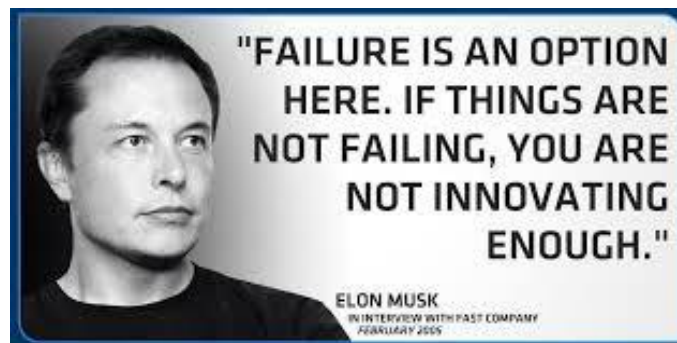




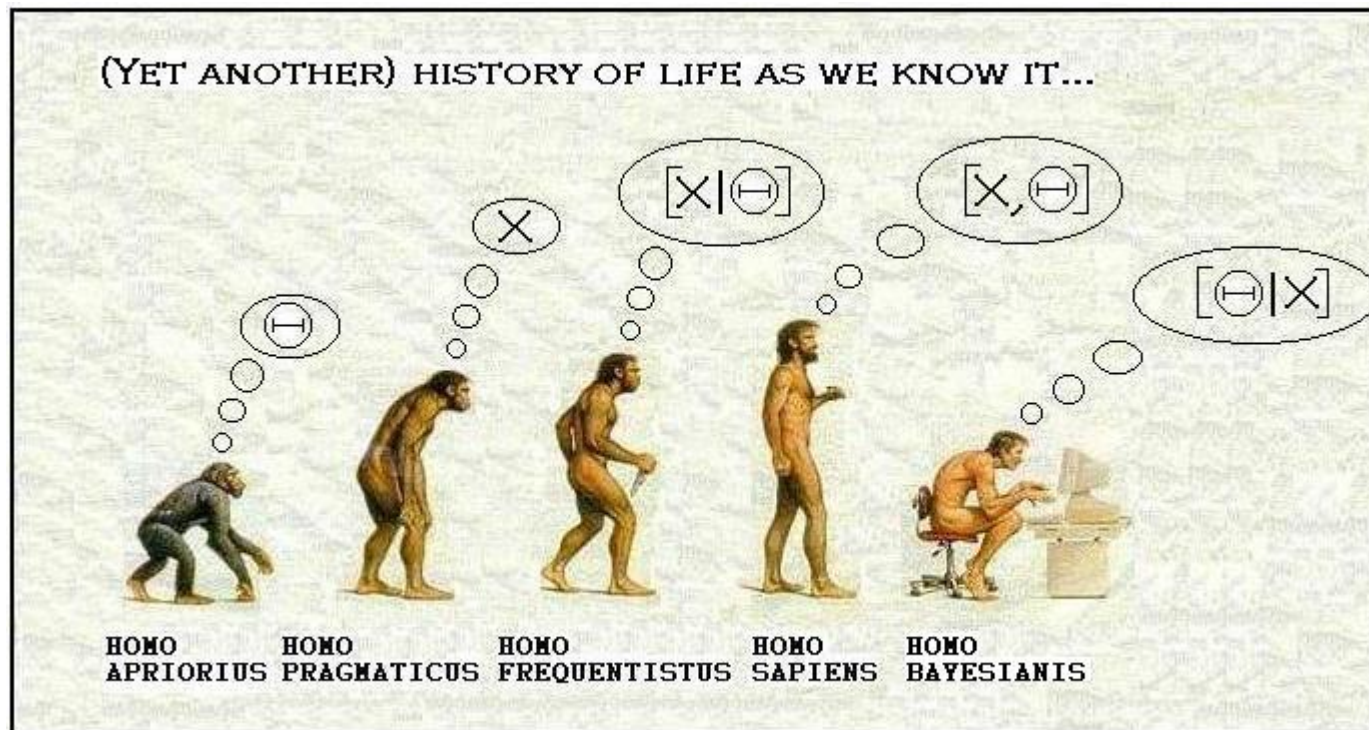
# Deep Reinforcement for Exotic Pricing

15/02/2019 //////////////

Olivier Croissant



# Parameters versus Facts



↑  
Illusion

↑  
Dogmatism

↑  
Scientific Procedure

↑  
Facts

↑  
Zenitude

# Energy Minimization



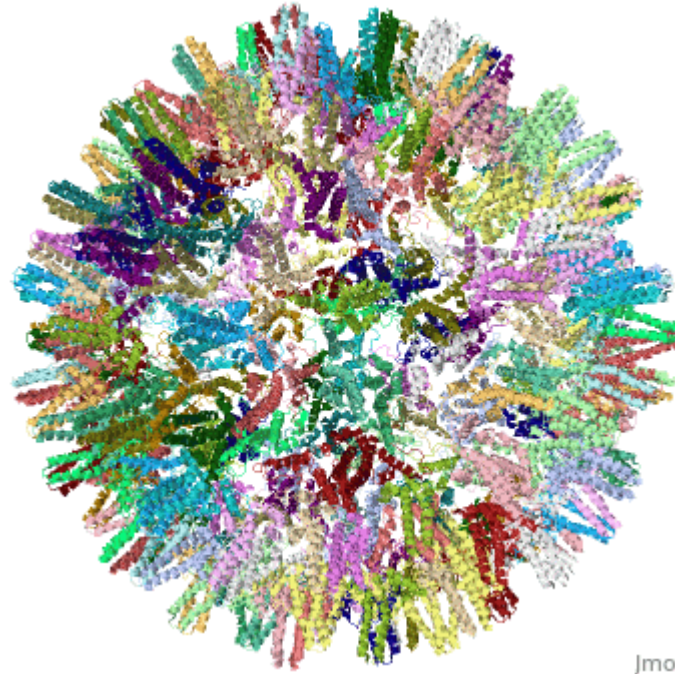
# 3D structure of a Coat protein



Structure of HIV-I coat protein hexamer (PDB entry [3gv2](#))  
**Virus coat proteins** (VCP) or capsid proteins coat the virus. It mediates the virial attachment to the host epithelial cell receptors



# Hepatitis B core/capsid protein (HBc)

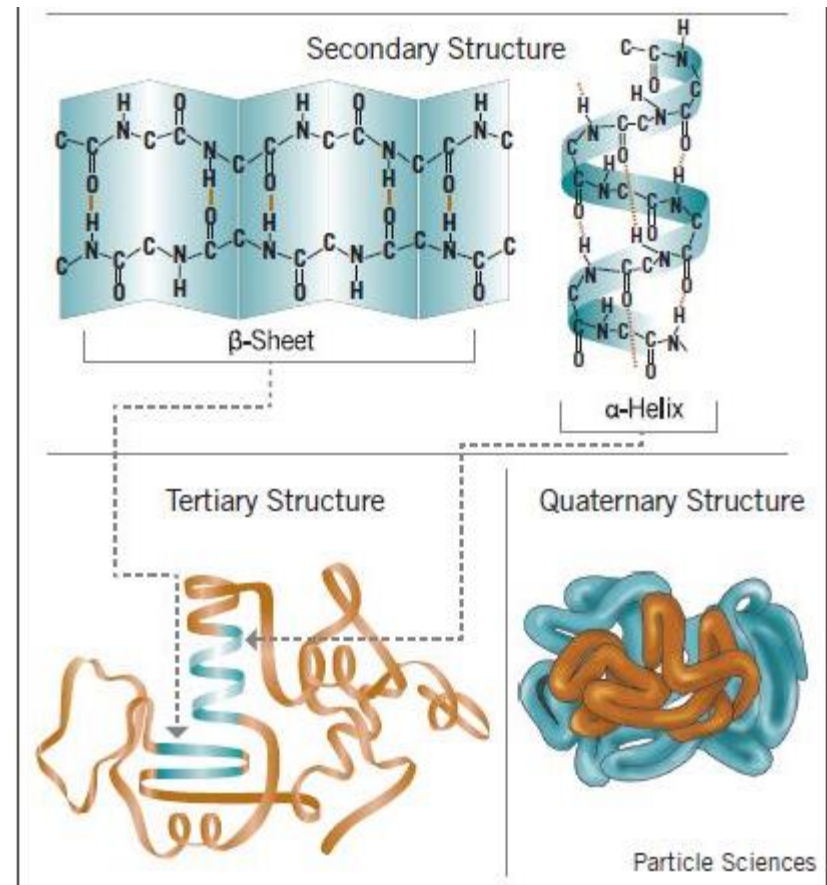
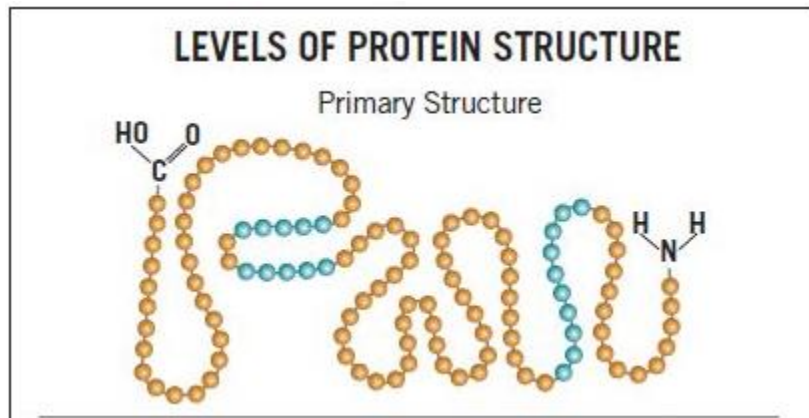


Jmol

**Hepatitis B core/capsid protein (HBc) in the viral life cycle  
major implication : HBc-targeting antivirals.**

**Hepatitis B is a small enveloped DNA virus that poses a major hazard  
to human health**

# Different classes of protein structure



# What is Deepmind



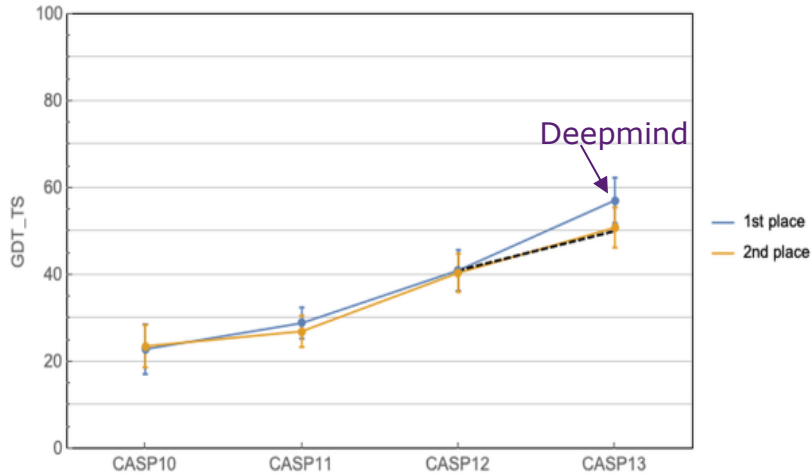
Demis Hassabis



On 26 January 2014, Google announced the company had acquired DeepMind for \$500 million



# AlphaFold wins CASP13 competition (2018)



It beat 98 competitors in the Critical Assessment of Structure Prediction (CASP) protein-folding competition in Cancun, where it successfully predicted the structure of 25 out of 43 proteins. (The second-place system could only predict three protein structures.)

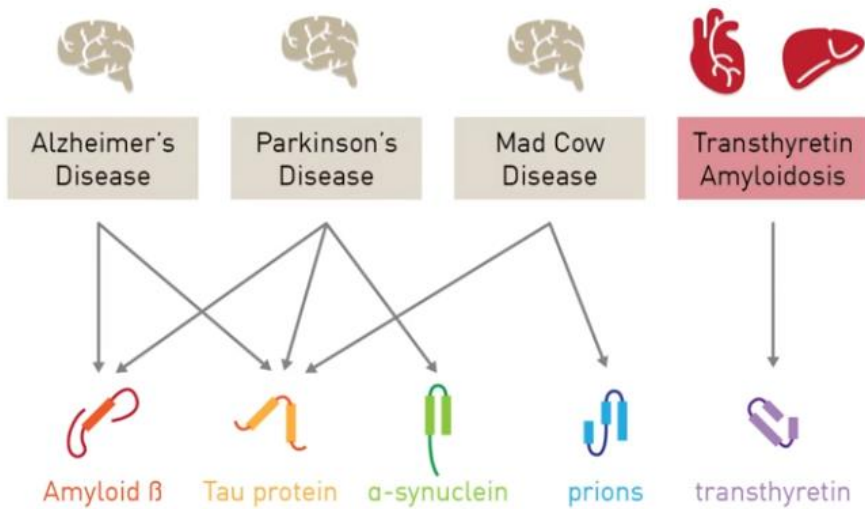
*Best and second best predictors at each CASP*

AlphaFold uses the entire distribution as a (protein-specific) statistical potential and minimizes of their predicted energy function

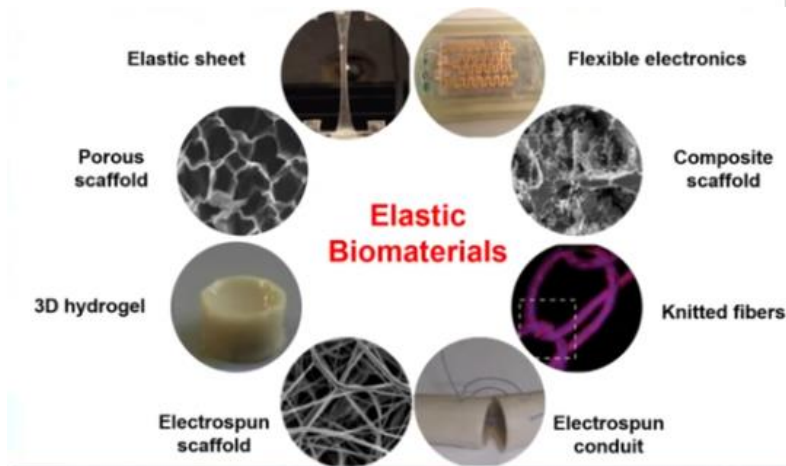
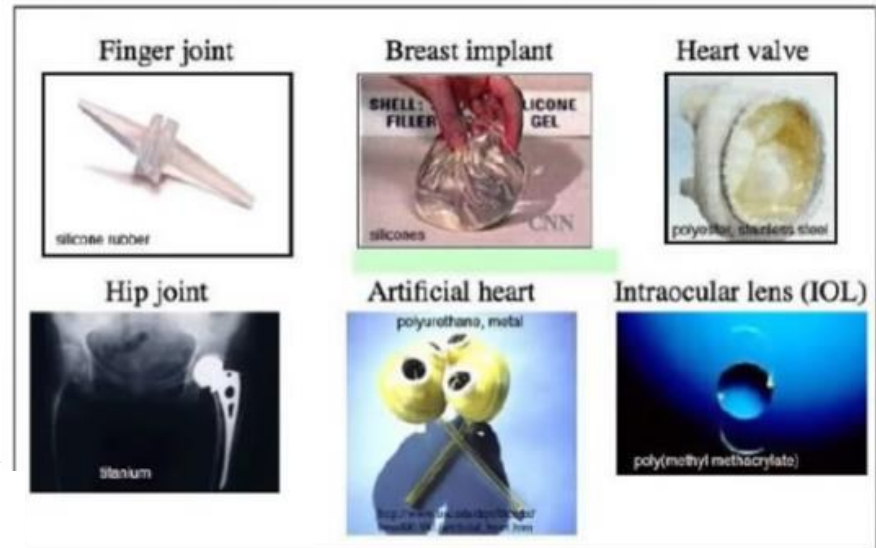
->More effective diagnoses and treatment of diseases such as Parkinson's and Alzheimer's, the DeepMind team noted, as these are believed to be caused by misfolded proteins. And it could aid in protein design, leading to protein-secreting bacteria that make wastewater biodegradable, for instance, and enzymes that can help manage pollutants such as plastic and oil



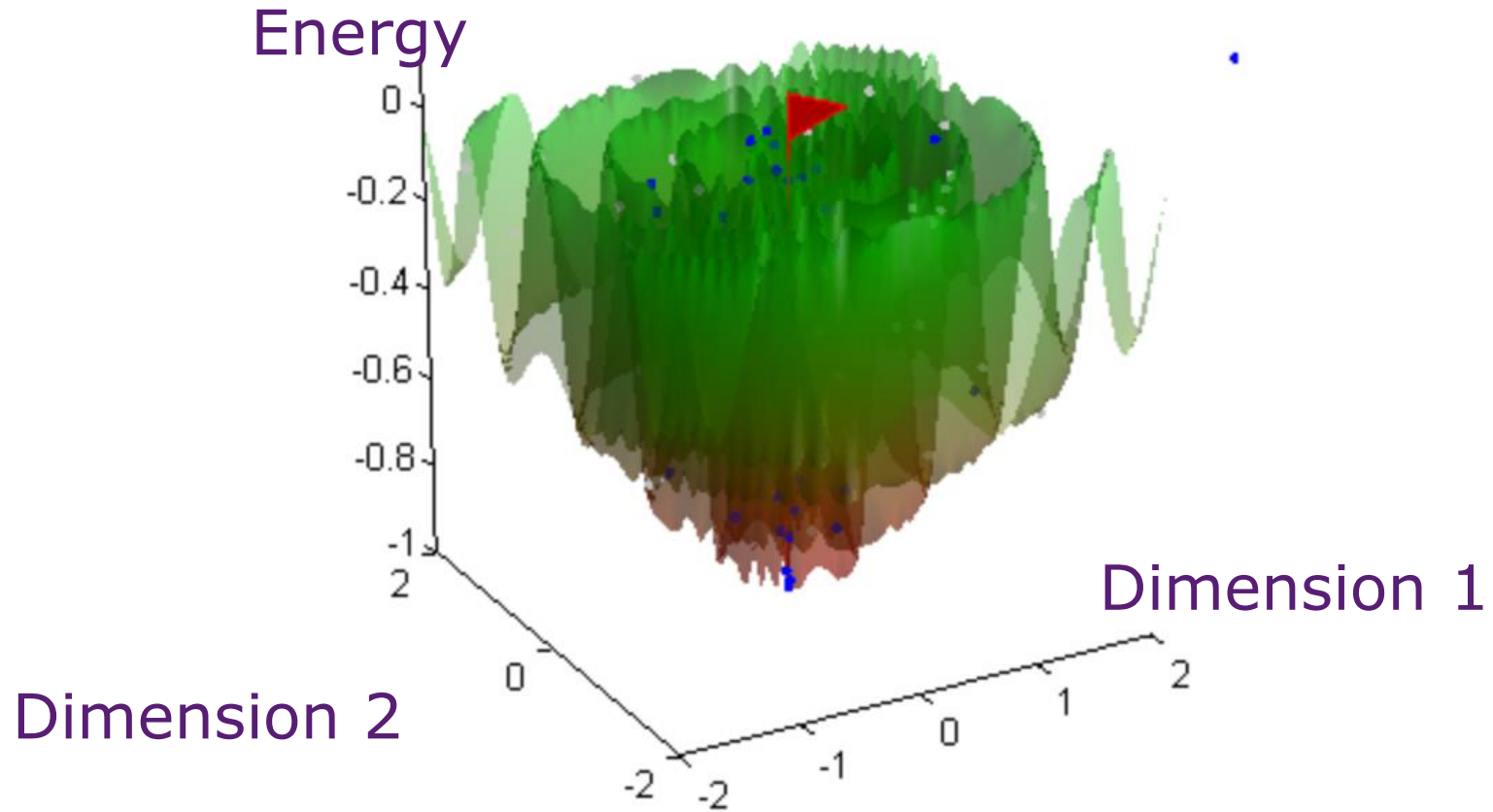
# Benefits of Protein Structure Mastering



## Biomaterials



# AlphaFold Minimization



# Yeti-Phoenix Fast Pricing



100K traject.

Black Sholes

3 underlyings

$$\begin{aligned}
 &\text{Bonus coupon} \quad \text{Yeti coupon} \quad \text{Phoenix coupon} \\
 &\text{Payoff}(t_i) = \{BO_i + \widehat{Y}E_i \mathbb{1}_{\{YB_i \leq P_i\}} + (PH_i) \mathbb{1}_{\{AB_i \leq P_i\}}\} \prod_{j=1}^{i-1} \mathbb{1}_{\{P_j < AB_j\}} + \\
 &\left[ \text{Gearing}(\delta_{type}(P_n - \text{Strike})^+) \left[ 1 - \prod_{k=0}^{n-1} \mathbb{1}_{\{DB_k < P_k\}} \right] \right] \prod_{i=1}^n \mathbb{1}_{\{P_i < AB_i\}} \\
 &\quad \uparrow \\
 &\quad \text{Put down and in}
 \end{aligned}$$

Where

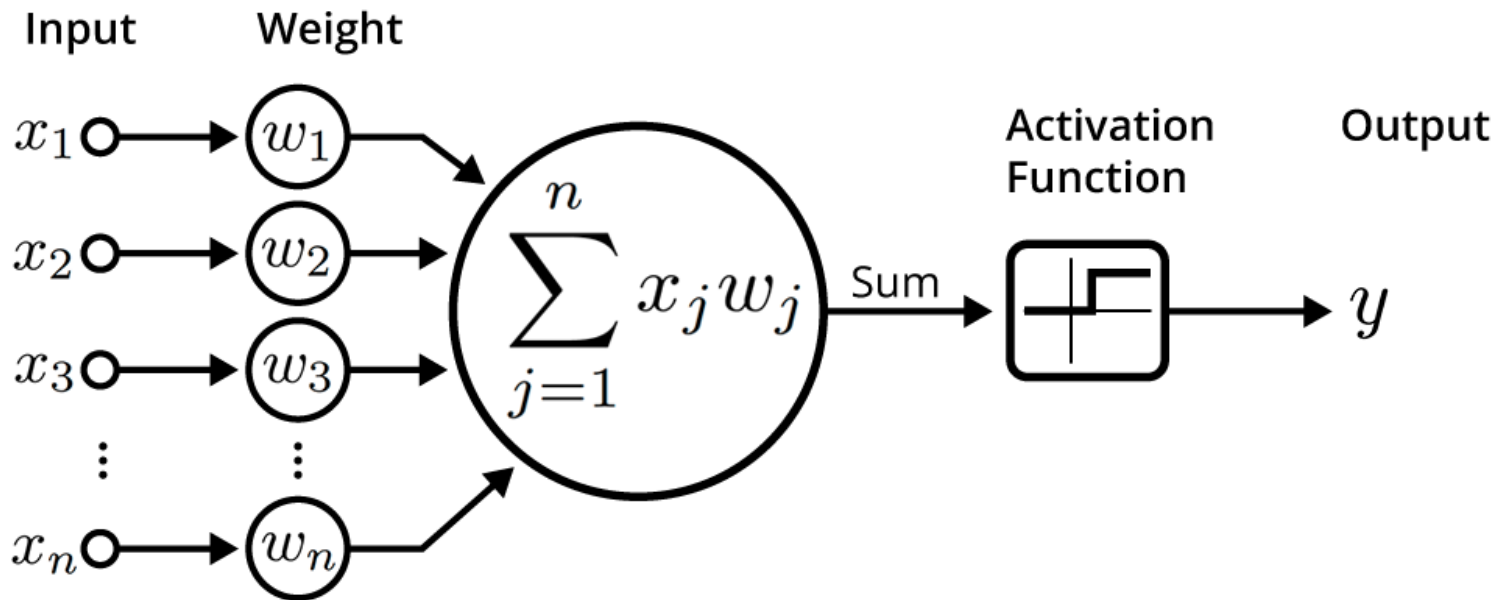
$$P_i = \text{Min}(S_1(t_i), S_2(t_i), S_3(t_i))$$



AutoCall

Worst ( 3 underlyings)

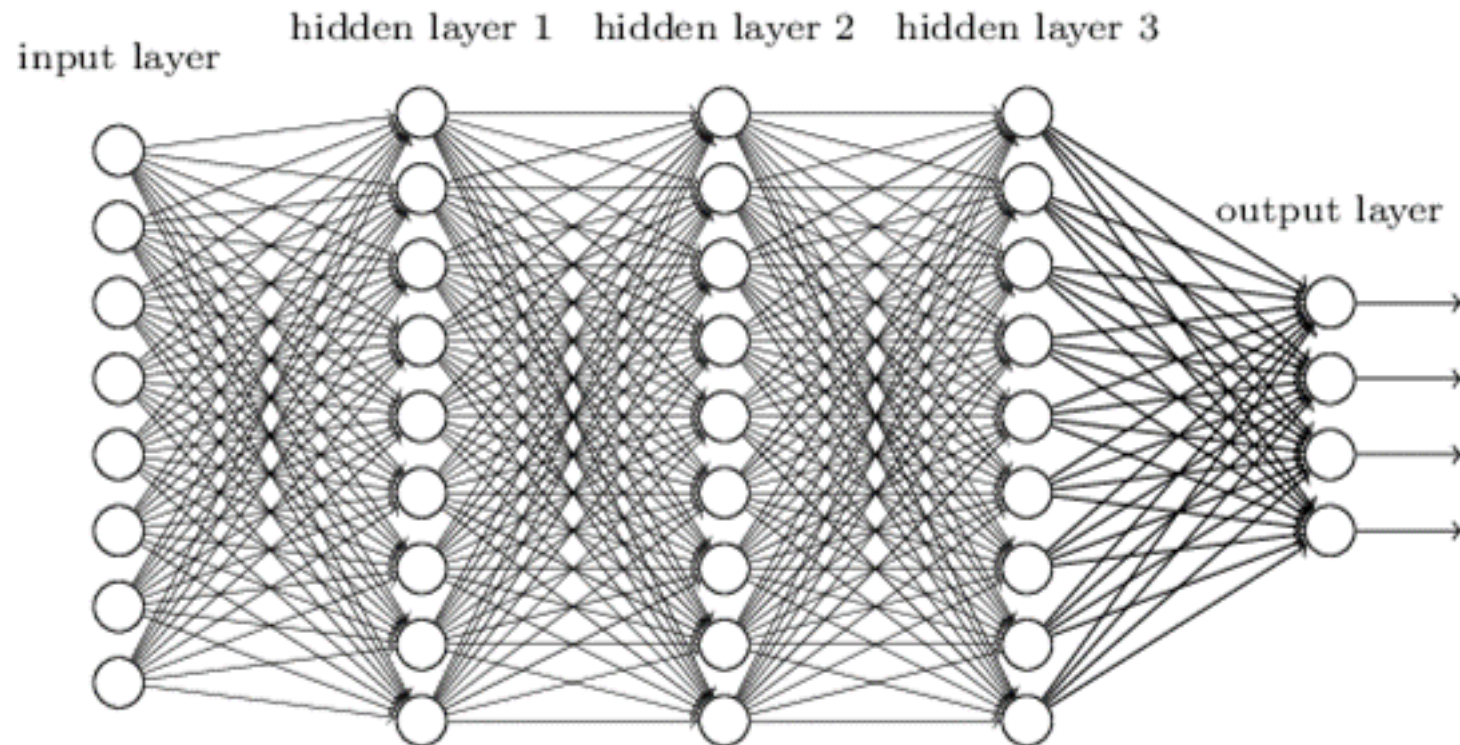
# A Neuron



An illustration of an artificial neuron. Source: Becoming Human.



# A Deep Neural Network



# Fast Pricing: Challenge

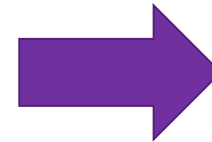


$x^i$ : 23 parameters: normalised

|     | M            | N           | O           | P              | Q             | R           | S            | T           | U            | V         |
|-----|--------------|-------------|-------------|----------------|---------------|-------------|--------------|-------------|--------------|-----------|
|     | bonus        | YetiBarrier | YetiCoupon  | PhoenixBarrier | PhoenixCoupon | PDIBarrier  | PDIGearing   | PDIStrike   | PDIType      | maturity  |
| 168 | 1,598662045  | 102,7934506 | 1,803550383 | 89,10621421    | 1,503284327   | 61,35308533 | -1,131986778 | 61,16504473 | -0,981872169 | 2,1633824 |
| 748 | -0,865687113 | 102,5289398 | 1,207499897 | 93,12703428    | 0,629783047   | 62,28204399 | 1,906473794  | 59,70435676 | 0,206153686  | 4,2224176 |
| 888 | 0,511093603  | 90,91612936 | 1,721972548 | 94,80305415    | 1,92341021    | 49,31570746 | -2,264827223 | 59,15778854 | 0,726806872  | 2,6569599 |
| 632 | 1,873411586  | 105,4550804 | 1,580305815 | 90,82321253    | 1,016549817   | 41,40652    | 2,741327755  | 67,40190344 | 0,587935765  | 4,3467833 |
| 925 | -0,024438552 | 102,3634059 | 1,858138975 | 97,85346983    | 0,802054037   | 43,9653955  | 1,853283181  | 51,66273998 | 0,489263864  | 2,8627828 |
| 774 | 0,968667823  | 106,7197266 | 0,120575059 | 91,6186545     | 0,509863802   | 65,75974463 | -1,767057384 | 61,62644528 | 0,463113878  | 1,6978381 |
| 827 | -1,859266393 | 101,6584001 | 1,15385071  | 88,00817664    | 1,251674194   | 53,84697645 | 4,660926601  | 43,58051936 | 0,952898331  | 1,2988642 |
| 558 | 1,38549049   | 100,2770735 | 0,874996886 | 90,33014487    | 1,088720604   | 68,02265178 | -0,962557176 | 57,20784498 | 0,011707592  | 2,9695811 |
| 014 | -1,609906608 | 98,63363177 | 0,459939613 | 88,59774229    | 1,681616988   | 48,9917834  | 4,847473595  | 69,79549342 | -0,506913407 | 3,0123358 |
| 902 | 0,212937624  | 94,98203874 | 0,143701045 | 91,98319076    | 1,063787317   | 60,32314953 | -1,005645658 | 63,8324717  | 0,530987777  | 3,8815812 |
| 393 | 0,286776665  | 105,1712089 | 0,628513979 | 83,83704042    | 1,252492095   | 55,76686052 | -0,286061138 | 53,32160594 | -0,304853521 | 2,1418366 |
| 555 | 1,826821038  | 97,03536388 | 0,930870132 | 91,3584631     | 0,94559383    | 47,00123052 | -2,243829941 | 56,50012468 | 0,000312713  | 2,2628155 |
| 117 | 1,648869155  | 109,9430667 | 0,818865586 | 90,69036591    | 0,984073955   | 51,52967911 | 4,705981603  | 41,04753564 | -0,091951987 | 1,2464048 |
| 668 | 1,797335297  | 91,58462697 | 1,321602856 | 86,86102174    | 1,494815143   | 67,82762906 | -4,088887842 | 67,759794   | -0,284710253 | 4,2824016 |

$y^i$ : price: normalized

| W    | X        | Y    |
|------|----------|------|
| ates | price    | forw |
| 13   | 28,15544 | 73,7 |
| 7    | 3,711043 | 67,8 |
| 19   | 33,21414 | 80,1 |
| 6    | 14,59038 | 77,5 |
| 18   | 16,84421 | 88,2 |
| 11   | 7,513129 | 83,0 |
| 20   | -0,06915 | 93,6 |
| 8    | 14,32587 | 73,3 |
| 10   | -2,54621 | 72,5 |
| 7    | 1,779381 | 75,4 |
| 12   | 7,629561 | 76,3 |
| 15   | 21,07478 | 84,8 |
| 21   | 11,57243 | 91,7 |
| 20   | 22,14178 | 79,7 |
| 5    | -0,19858 | 111, |



Name of the Game

Non-Linear Interpolation in 23 Parameters

Learning Base: 1 M Samples

Testing Base: 400 K Samples

$$risk = \sum_{i \in \text{Testing Base}} (y_{predicted}^i - y_{base}^i)^2$$

# Challenge Natixis - ENS



 NATIXIS



## Dates

From Jan. 1, 2019 to Jan. 1, 2020

## Challenge context

Among capital markets' activity spectrum, the Equity Derivatives department creates investment, hedging and multipurpose solutions for professional clients. These solutions use exotic options based on one or more baskets of underlyings among which stocks, indices and other risky vehicles. Those exotic options need complex numerical algorithms to be priced, using Monte-Carlo methods in a non-trivial way due to the path-dependent nature of the pay-off and the callability of the instrument. The typical time of a single pricing can go between a few seconds and a few minutes on a farm of servers.

[natixis\\_squared\\_error](#)  
Download metric file

Related courses  
Big Data et ses applications  
Apprentissage par réseaux de  
42 challenge  
Apprentissage Statistique ECN

Files given to participants

[x\\_train](#)  
input data of the training set  
[y\\_train](#)  
output data of the training set  
[x\\_test](#)  
input data of the testing set  
[supplementary files](#)  
data-readers, baseline scripts, instru

The provider

Introduction

Public ranking

Final ranking

Custom ranking

Sta

23/04/2019

*This is the public ranking which evolve during the entire season. We use the public se*

| Ranking | Date                      | User(s)        | Score  |
|---------|---------------------------|----------------|--------|
| 1       | April 20, 2019, 8:21 a.m. | Cmen           | 0.0478 |
| 2       | March 18, 2019, 4:09 p.m. | Soumirai       | 0.0649 |
| 3       | April 13, 2019, 4:46 p.m. | hazevedochaves | 0.0763 |
| 4       | March 18, 2019, 1:33 p.m. | dofuer99       | 0.1052 |
| 5       | -                         | benchmark      | 0.1146 |
| 6       | April 3, 2019, 8:49 p.m.  | aminek         | 0.2049 |
| 7       | April 3, 2019, 7:01 p.m.  | toto961        | 0.2049 |
| 8       | April 2, 2019, 8:53 p.m.  | JCc            | 0.2088 |
| 9       | April 3, 2019, 3:37 p.m.  | wisniewski.w   | 0.2786 |
| 10      | Feb. 20, 2019, 8:46 a.m.  | Zijun_Yao      | 0.3096 |

Second Benchmark : 0,053

# Benchmark: Methodology



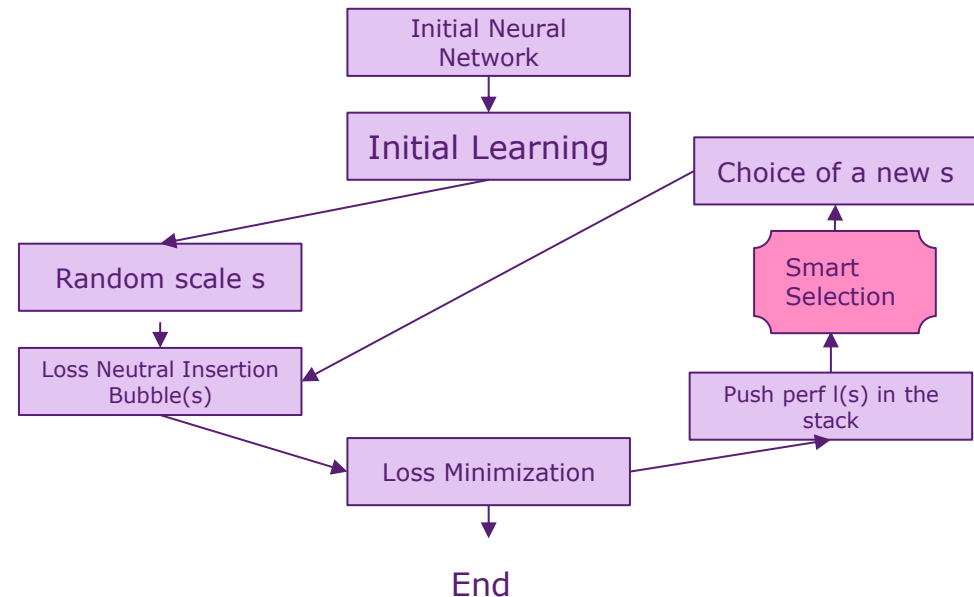
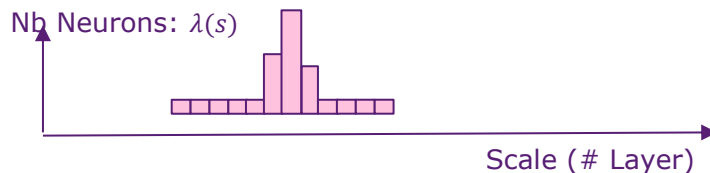
## Neural network in Keras/ Tensorflow

### 11 layers (Dense)

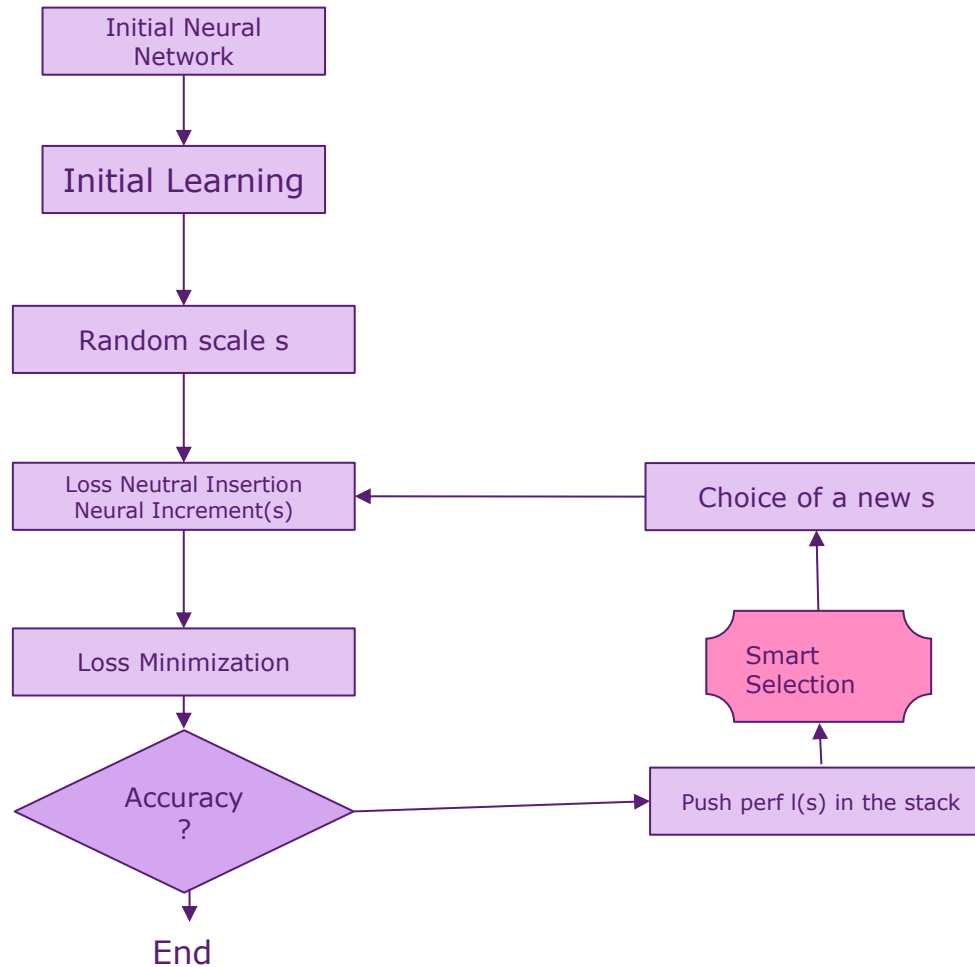
Dynamic building of the network: genetic algorithm  
To help with local minimum handling

#### Initial Learning

```
model = Sequential()
model.add(Dense(n1=params.NB_NEURON_PRINCIPAL, input_dim=params.INPUT_DIM, activation=params.ACTIVATION_PRINCIPALE))
model.add(Dense(n2=params.NB_NEURON_PRINCIPAL, activation=params.ACTIVATION_PRINCIPALE))
model.add(Dense(n3=params.NB_NEURON_PRINCIPAL, activation=params.ACTIVATION_PRINCIPALE))
model.add(Dense(n4=params.NB_NEURON_PRINCIPAL, activation=params.ACTIVATION_PRINCIPALE))
model.add(Dense(n5=params.NB_NEURON_PRINCIPAL, activation=params.ACTIVATION_PRINCIPALE))
model.add(Dense(n6=params.NB_NEURON_PRINCIPAL, activation=params.ACTIVATION_PRINCIPALE))
model.add(Dense(n7=params.NB_NEURON_PRINCIPAL, activation=params.ACTIVATION_PRINCIPALE))
model.add(Dense(n8=params.NB_NEURON_PRINCIPAL, activation=params.ACTIVATION_PRINCIPALE))
model.add(Dense(n9=params.NB_NEURON_PRINCIPAL, activation=params.ACTIVATION_PRINCIPALE))
model.add(Dense(1, activation=params.ACTIVATION_PRINCIPALE_FINAL))
model.compile(loss='mse', optimizer='adam', metrics=['accuracy'])
```

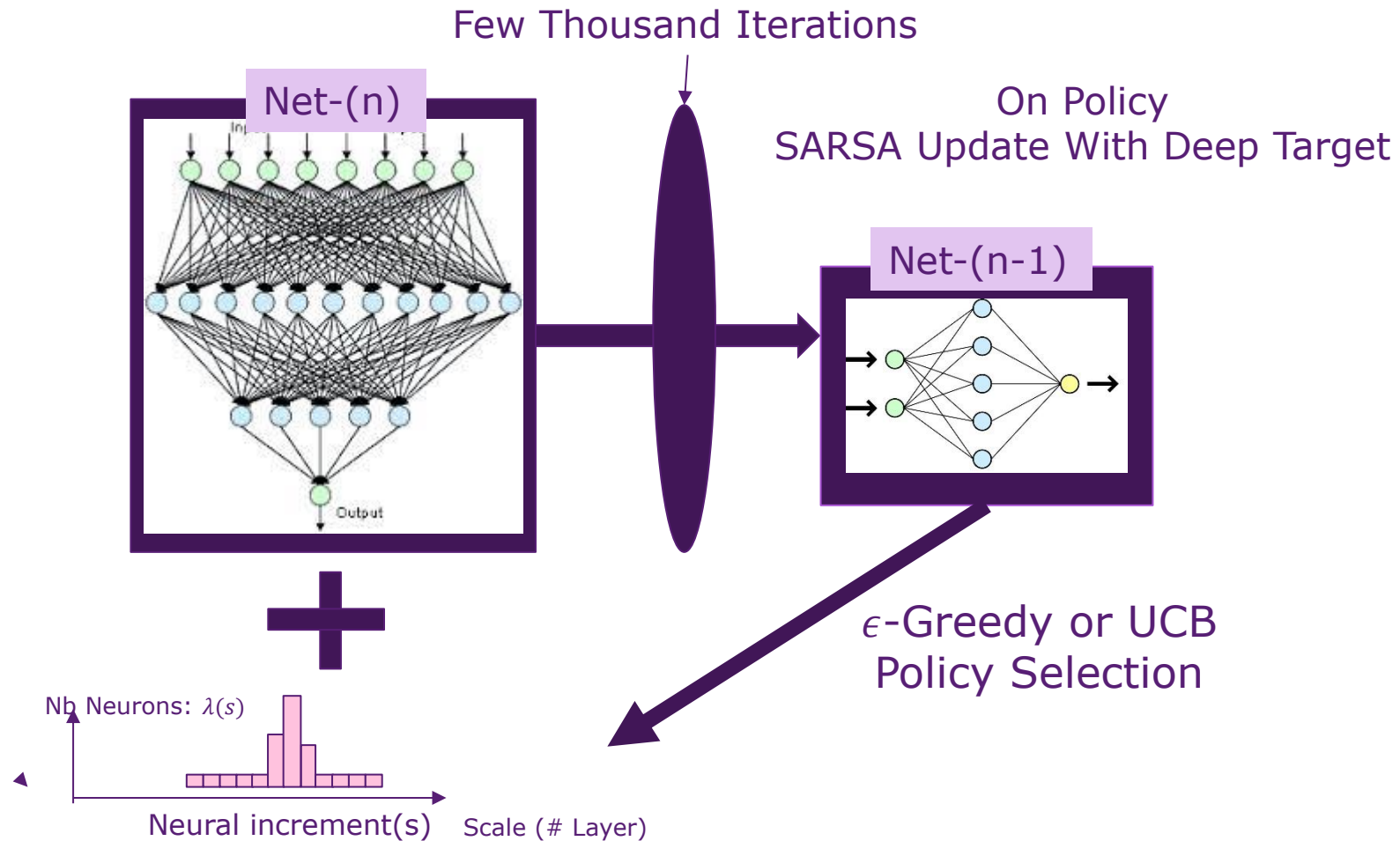


# Benchmark: Methodology

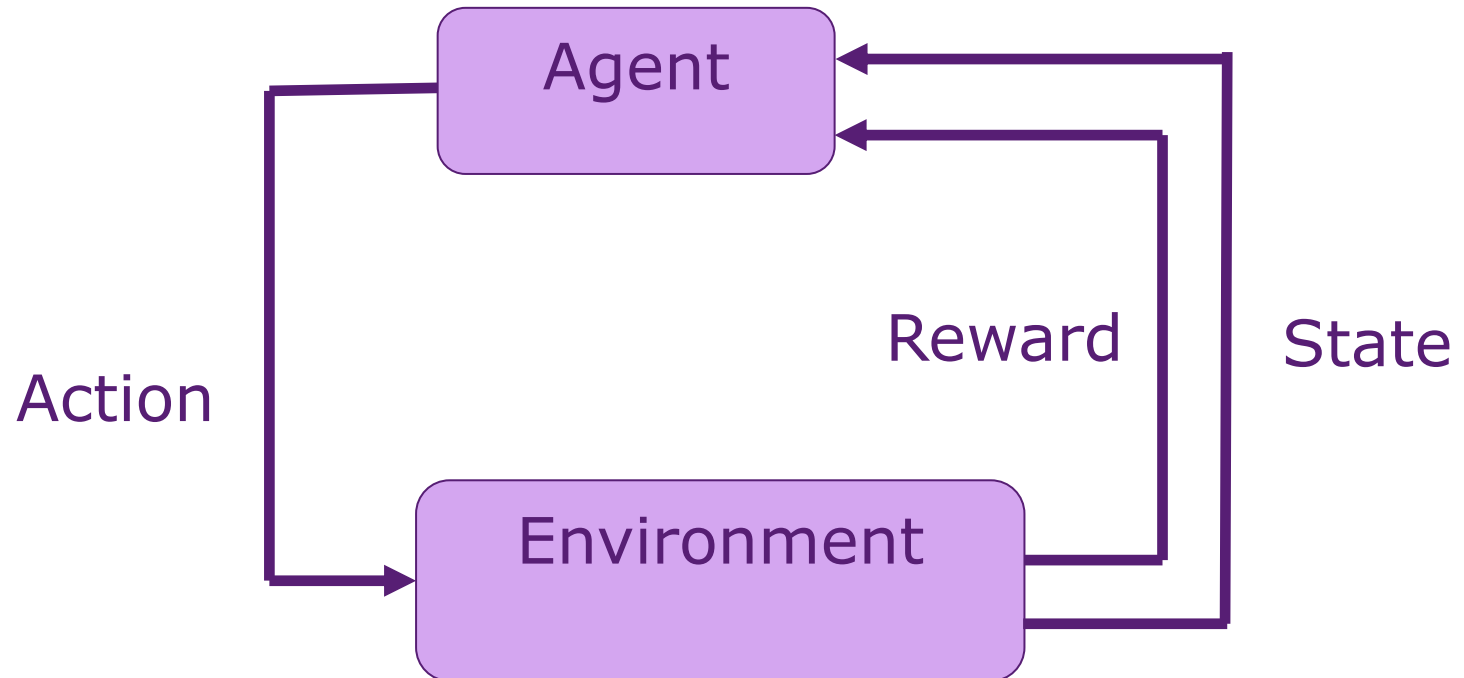




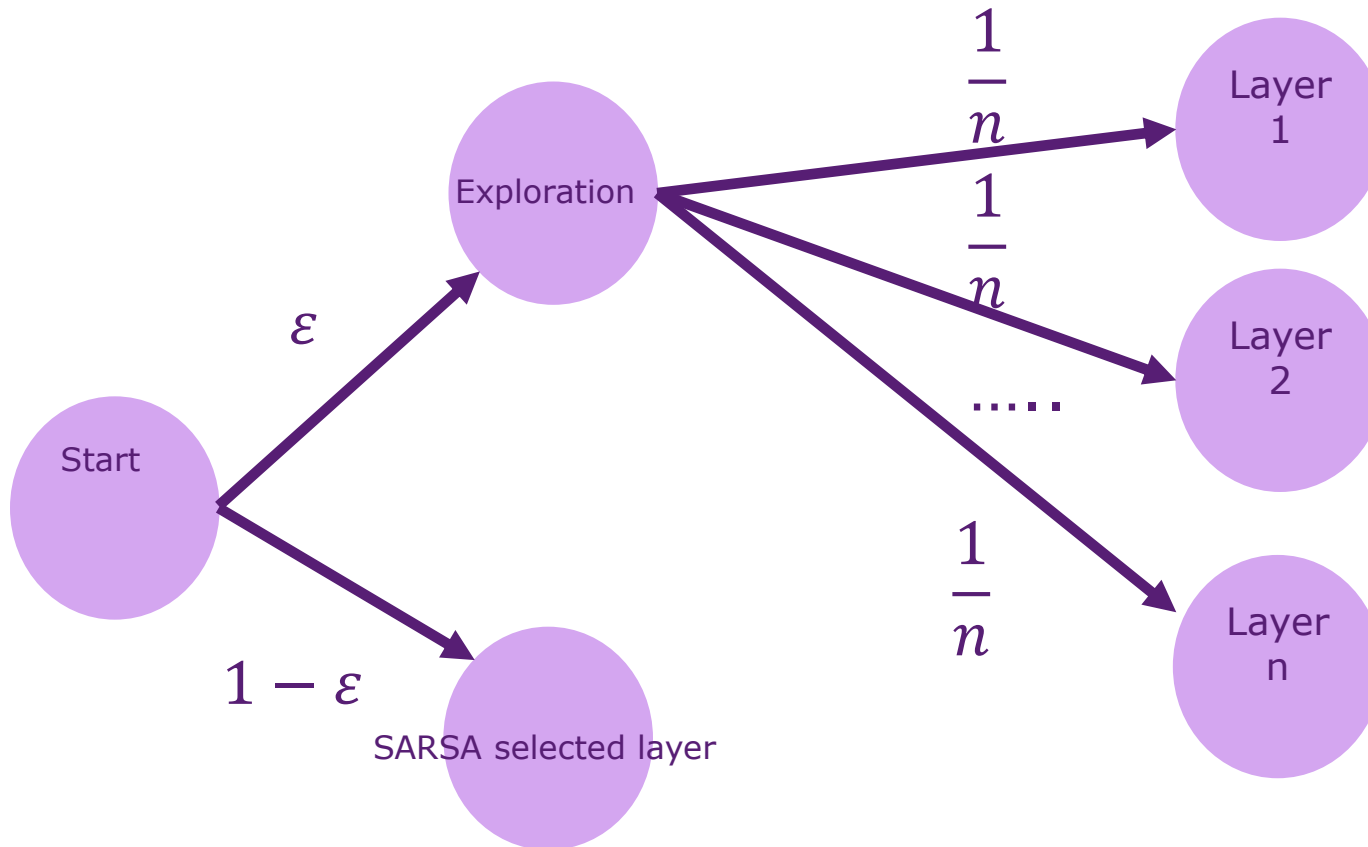
# Deep Learning with deep TD-Learning



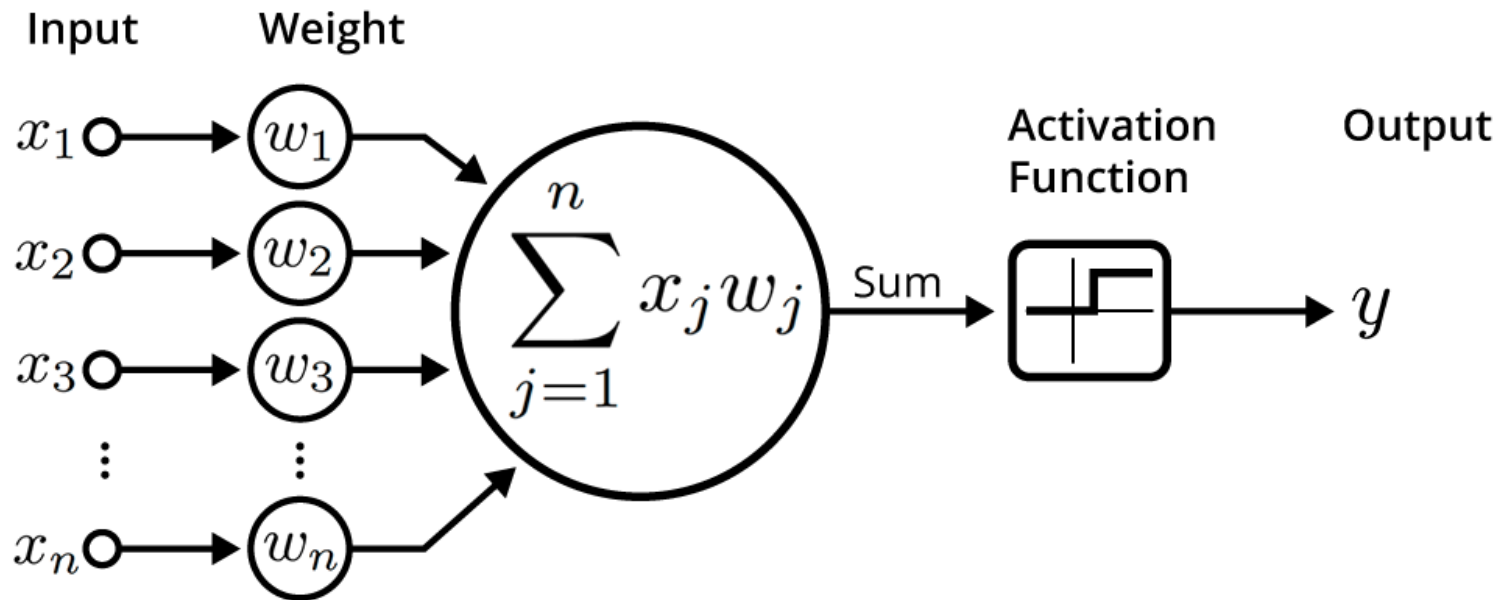
# Markov Decision Process



# Epsilon greedyness

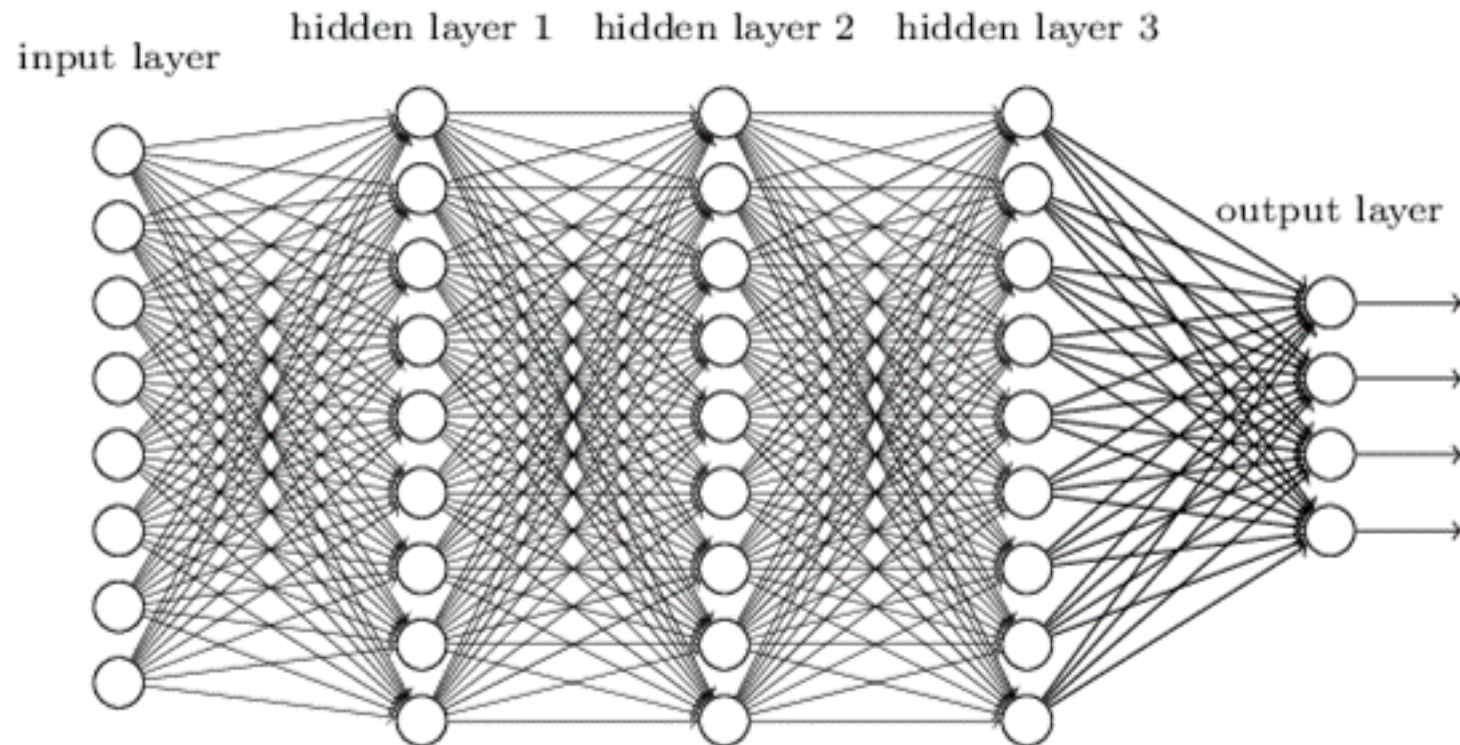


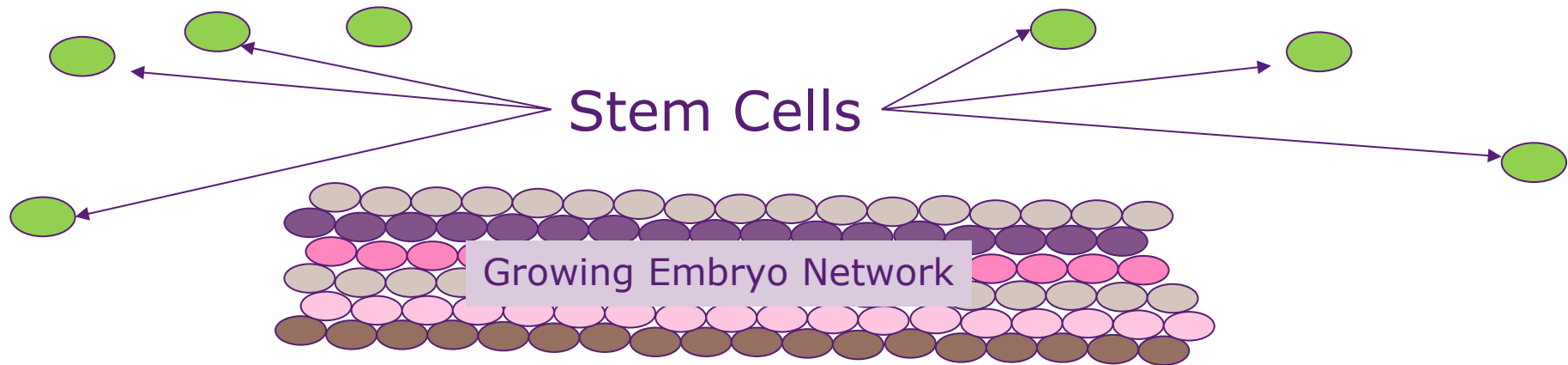
# A Neuron



An illustration of an artificial neuron. Source: Becoming Human.

# A Deep Neural Network

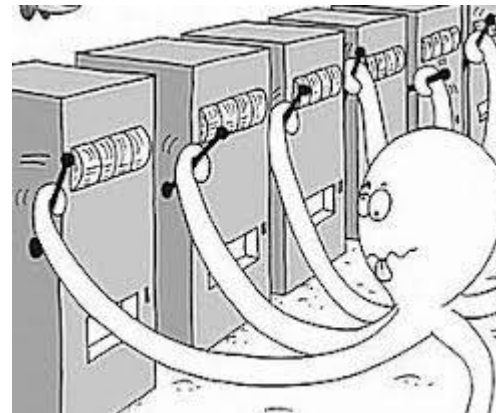




$$Q\left\{N + \sum_{i=1}^n b_i\right\} \approx Q\{N\} + \sum_{i=1}^n (Q\{N + b_i\} - Q\{N\}) = \sum_{i=1}^n r_i$$

## Multi-armed bandit

$r_i$  Accuracy increase at step  $i$





# 3 Reinforcement solutions with Q-value



| Policy       | Return               |
|--------------|----------------------|
| Chosen Scale | Performance Increase |
| 1            | 0,00                 |
| 5            | 0,001                |
| 7            | 0,005                |
| 9            | 0,001                |
| 10           | 0,00                 |
| 4            | 0,00                 |
| 8            | 0,003                |

| State                   | Policy       | Return               |
|-------------------------|--------------|----------------------|
| Weights: Complete State | Chosen Scale | Performance Increase |
| W1,..Wn                 | 1            | 0,00                 |
| W1,..Wn                 | 5            | 0,001                |
| W1,..Wn                 | 7            | 0,005                |
| W1,..Wn                 | 9            | 0,001                |
| W1,..Wn                 | 10           | 0,00                 |
| W1,..Wn                 | 4            | 0,00                 |
| W1,..Wn                 | 8            | 0,003                |

| State       | Policy       | Return               |
|-------------|--------------|----------------------|
| performance | Chosen Scale | Performance Increase |
| 0,93        | 1            | 0,00                 |
| 0,94        | 5            | 0,001                |
| 0,941       | 7            | 0,005                |
| 0,946       | 9            | 0,001                |
| 0,946       | 10           | 0,00                 |
| 0,946       | 4            | 0,00                 |
| 0,949       | 8            | 0,003                |

## Multi-Armed Bandit

## Full-Scale Reinforcement

## POMDP Reinforcement

Reinforcement learning can use  $TD(\lambda)$ -learning

Q-value  $q(\text{state}, a(k_{scale}))$ : Expected value for the accuracy after learning with the bubble associated with the scale  $k_{scale}$ .

$Q(\text{state}, k_{scale}, w)$  : Deep action out Q-value (deep neural network)

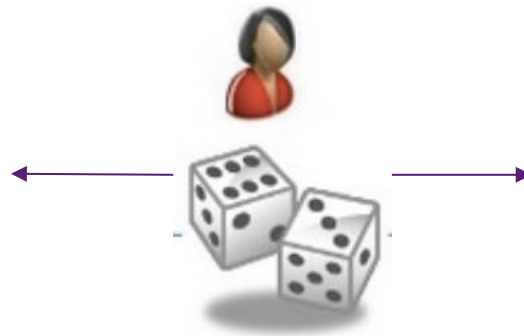
# Epsilon-Greedy Policy Selection



With Probability  $\epsilon$

Exploration

Try a  
Random  
Layer



With Probability  $1-\epsilon$

Exploitation

Greedy  
Policy

Concentrate  
On the Best  
Layer

# From Bellman Equation to Policy Search



If we know the policy :

- Bellman equation
- Value Iteration
- Policy Evaluation

$$V^\pi(s) \leftarrow \sum_{s'} P(s'|s, \pi(s)) (R(s, \pi(s), s') + \gamma V^\pi(s'))$$

We look for the best policy :

$$\left( \begin{array}{l} V^*(s) \\ a^* = \pi(s) \end{array} \right) \leftarrow \max_a \sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma V^*(s'))$$

# From Policy Search to Q-Learning



We split the policy search in two steps:

$$\left( \begin{array}{l} V^*(s) \\ a^* = \pi(s) \end{array} \right) \leftarrow \max_a \sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma V^*(s'))$$

We introduce the Q-function:

$$Q^*(s, a) \leftarrow \sum_{s'} P(s'|s, a) \left( R(s, a, s') + \gamma \max_{a'} Q_{k-1}(s', a') \right)$$

$$V^*(s) = \max_{a'} Q^*(s, a')$$

Non-Linear

# From Q-Learning to SARSA



## Off Policy : Q learning

$$Q_k(s, a) \leftarrow \sum_{s'} P(s'|s, a) \left( R(s, a, s') + \gamma \max_{a'} Q_{k-1}(s', a') \right)$$

## On Policy : SARSA

$$Q_k(s, a) \leftarrow \sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma Q_{k-1}(s', a'))$$

Linear

Need a policy here !

# SARSA Strategy



1) Determine the Q value  $\hat{Q}_t(s_t, a, \mathbf{w})$  :

“Deep out Q action-value”

Goal of the Strategy

2) Determine the policy (layer) by :

$$a_t = \operatorname{argmax}_{a \in \mathcal{A}} \{ \hat{Q}_t(s_t, a, \mathbf{w}) \}$$



# Deep SARSA Reinforcement



We want to minimize :

We assume it is known here

Neural Network

Oracle

Layer Number

$$J(w) = E_{\pi} \left[ \left( G_t - Q(s_t, k_t, w) \right)^2 \right]$$

State

Gradient Policy

Neural Net Weights and Biases

SGD : Update of the weights

$$\Delta w = \alpha (G_t - Q(s_t, k_t, w)) \nabla_w Q(s_t, k_t, w)$$

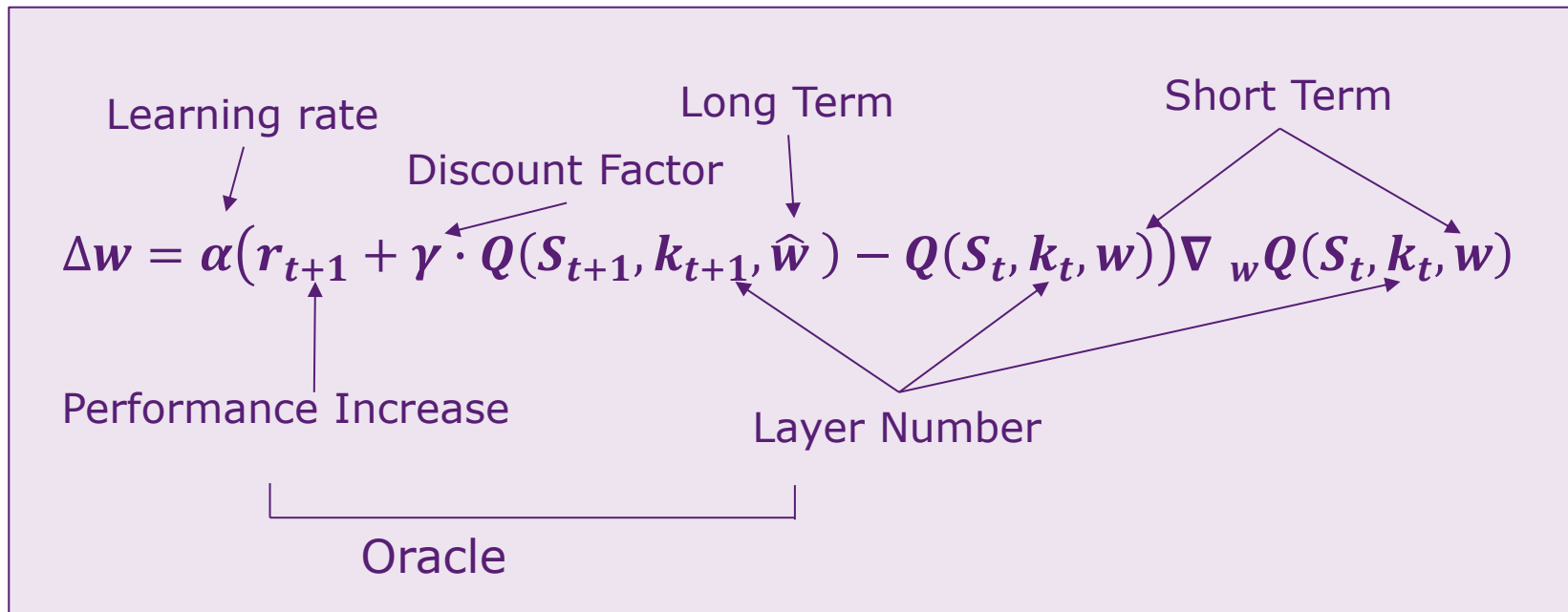
The oracle : The best accuracy  $G_t = \sum_{\tau=1}^{\infty} r_{\tau}$

$r_{\tau}$  : Improvement of the loss after N iterations

# SARSA : Fixing the Oracle



Update : 
$$\Delta \mathbf{w} = \alpha (G_t - Q(\mathbf{S}_t, \mathbf{k}_t, \mathbf{w})) \nabla_{\mathbf{w}} Q(\mathbf{S}_t, \mathbf{k}_t, \mathbf{w})$$



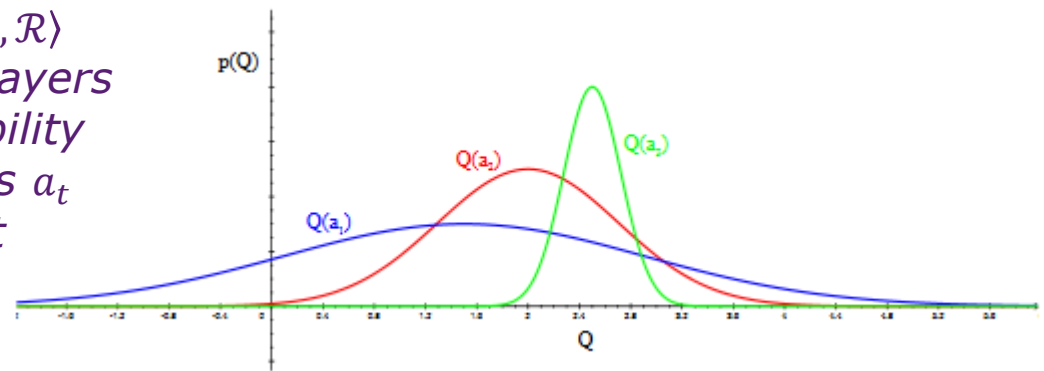
⇔ Making a predictive model For the dynamic

# The Multi-Armed Bandit Problem



## Exploration vs Exploitation

*A multi-armed bandit is a tuple  $\langle \mathcal{A}, \mathcal{R} \rangle$   
 $\mathcal{A}$  is a set of action, here a set of layers  
 $\mathcal{R}^a(r) = \mathbb{P}[r|a]$  is an unknown probability  
At each time step the agent selects  $a_t$   
At each time step the environment  
generates a reward  $r_t \sim \mathcal{R}^a$   
The goal is to maximize  $\sum_{\tau=1}^t r_\tau$*



Which action should we pick ?

The more uncertain we are about an action-value  
The more important it is to explore that action

It could turn to be the best action

# Lower Bound on Total Regret



## Theorem (Lai and Robbins)

***Asymptotic total regret is at least logarithmic in number of steps***

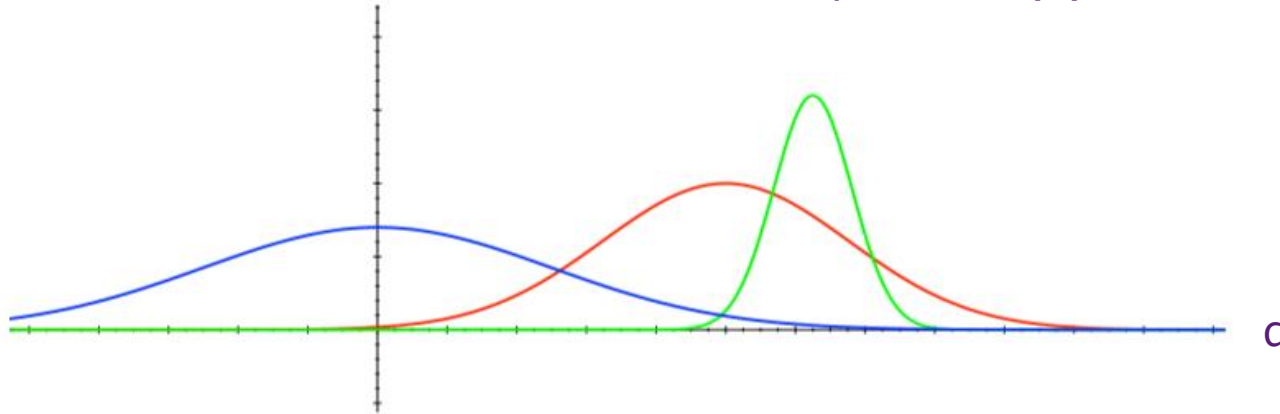
$$\lim_{t \rightarrow \infty} L_t \geq \log(t) \sum_{a | \Delta_a > 0} \frac{\Delta_a}{KL(\mathcal{R}^a || \mathcal{R}^{a^*})}$$

**Where the gap of action  $a$  is  $\Delta_a = V^* - Q(a)$**

# Optimism in Face of Uncertainty



Historical distribution of the uncertain q-value  $Q(a)$



After picking **red** action

We less uncertain about the value

An more likely to pick another action

Until we home in on best action

Estimate an upper confidence  $\hat{u}_t(a)$  for each action value

Such that  $Q(a) < \hat{Q}_t(a) + \hat{u}_t(a)$  with high probability

Then we select an action maximizing Upper Confidende Bound

$$a_t = \operatorname{argmax}_{a \in \mathcal{A}} \{ \hat{Q}_t(a) + \hat{u}_t(a) \}$$

# Upper Confidence Bounds



## Theorem (Hoeffding's Inequality)

Let  $X_1, X_2, \dots, X_t$  be i.i.d random variables in  $[0,1]$  and let  $\bar{X}_t = \frac{1}{t} \sum_{\tau=1}^t X_\tau$  be the sample mean; Then

$$\mathbb{P}[\bar{X}_t > \mathbb{E}[X] + u] \leq e^{-2tu^2}$$

Applied to our case (conditioning on a)

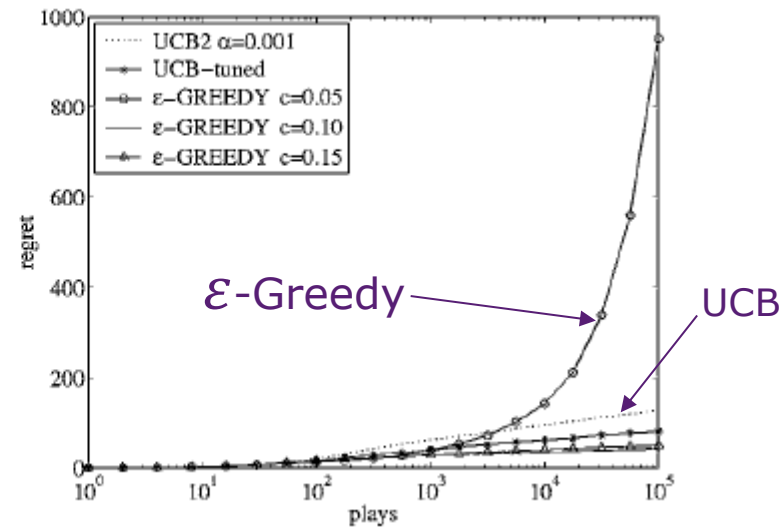
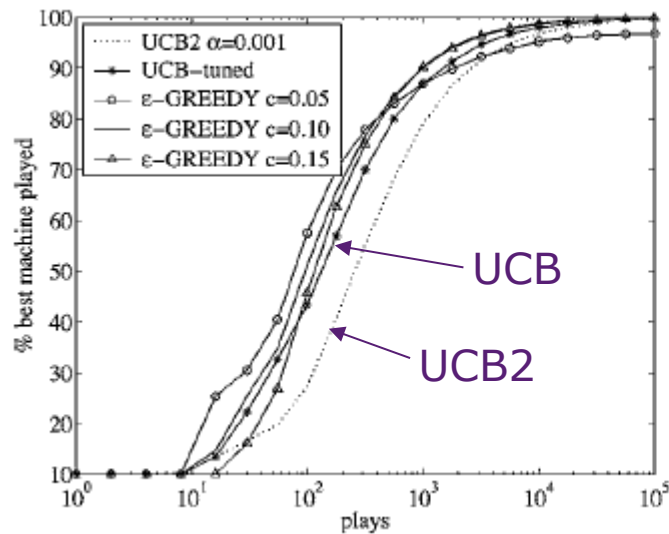
$$\mathbb{P}[Q(a) > \widehat{Q}_t(a) + \mathcal{U}_t(a)] \leq e^{-2\mathcal{N}_t(a)\mathcal{U}_t(a)^2} \quad p = t^{-4}$$

Where the count  $\mathcal{N}_t(a)$  is the number of selection of the action a

$$\mathcal{U}_t(a) = \sqrt{\frac{2\log(t)}{\mathcal{N}_t(a)}}$$

Asymptotically  
Optimal !

# Comparaison UCB vs $\epsilon$ -Greedy Strategies





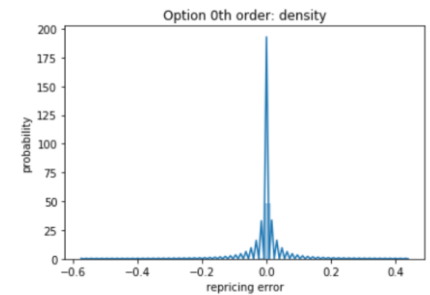
# Benchmark Performances



## Benchmark 1

| Probability Level | Lower Bound | Higher Bound |
|-------------------|-------------|--------------|
| 0,001             | -0,11816    | 0,0567       |
| 0,01              | -0,02981    | 0,00179      |
| 0,02              | -0,01091    | 0,00000      |
| 0,03              | -0,00144    | 0,00000      |
| 0,04              | 0,00000     | 0,00000      |

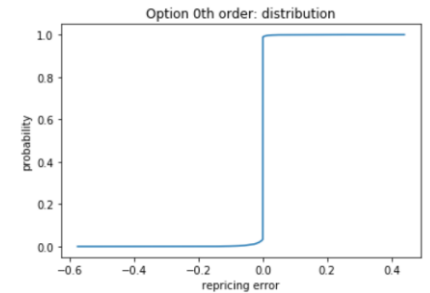
96% of the case  
=  
Less than 1Bp error  
Note Challenge : 0,11



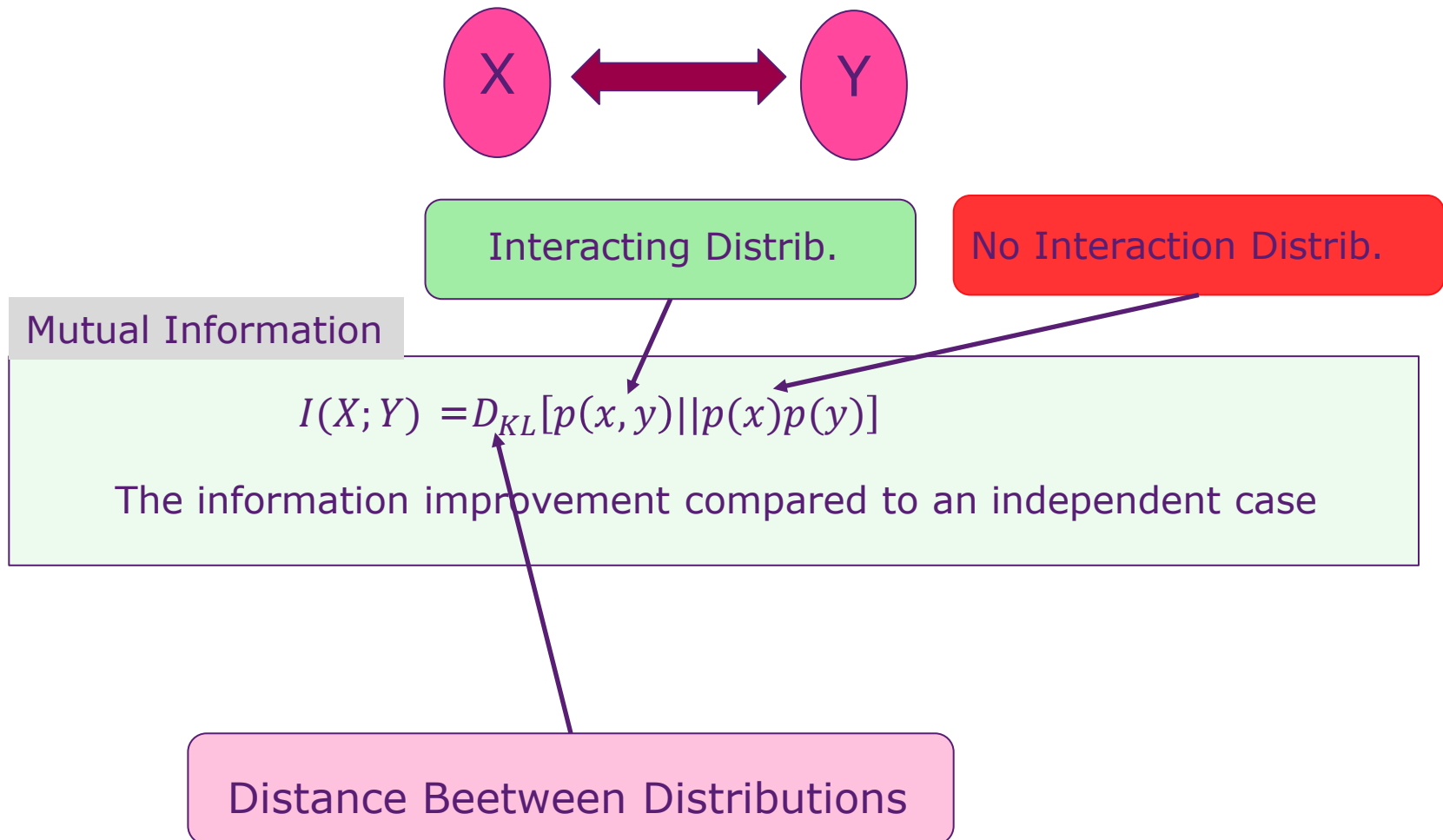
## Benchmark 2

| Probability Level | Lower Bound | Higher Bound |
|-------------------|-------------|--------------|
| 0,001             | -0,06118    | 0,0298       |
| 0,01              | -0,01572    | 0,00067      |
| 0,02              | -0,00094    | 0,00000      |
| 0,03              | 0,00000     | 0,00000      |
| 0,04              | 0,00000     | 0,00000      |

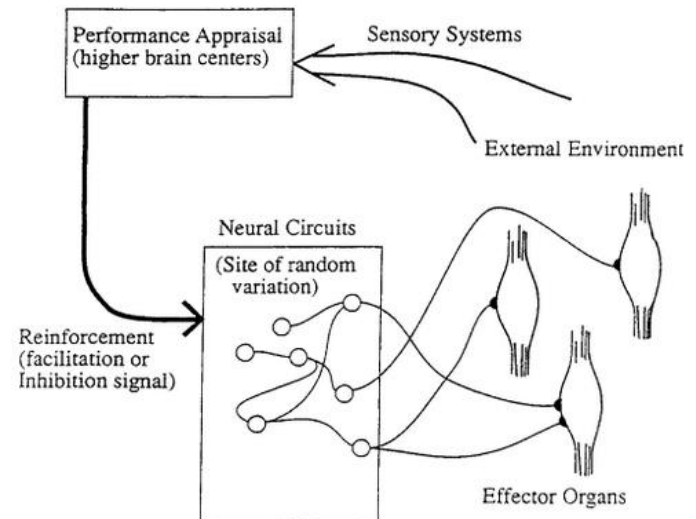
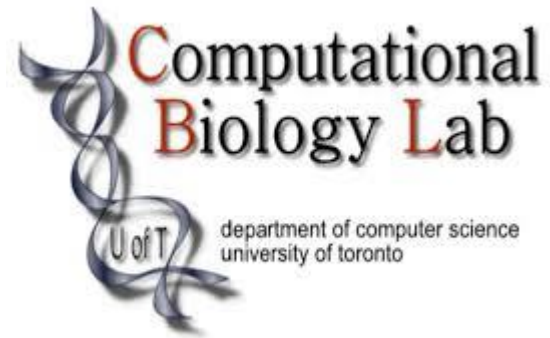
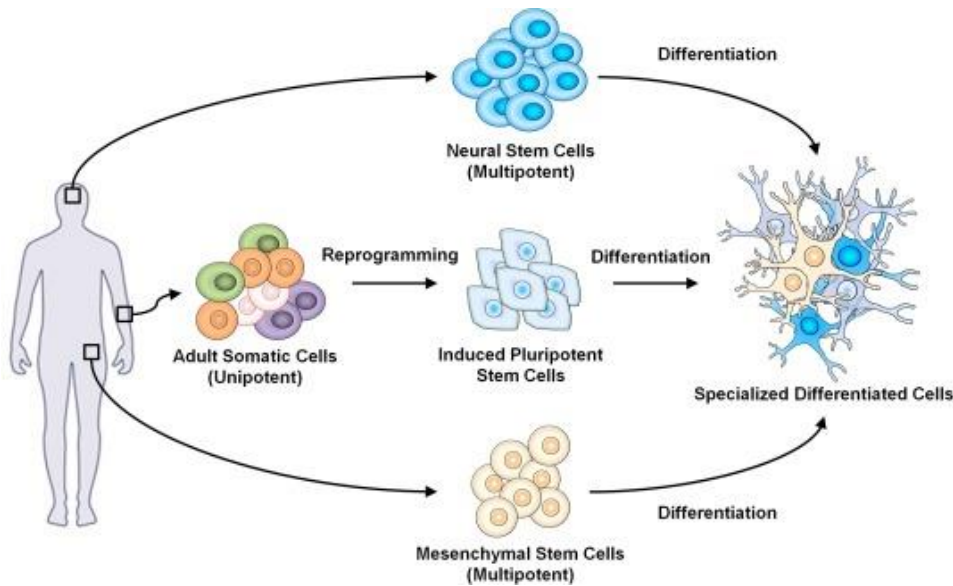
97% of the case  
=  
Less than 1Bp error  
Note Challenge : 0,053



# Interactions Between Layers

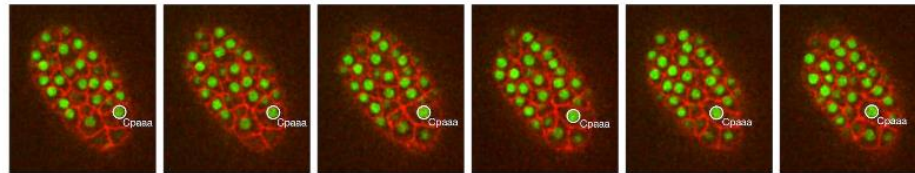


# Reinforcement Creates Life

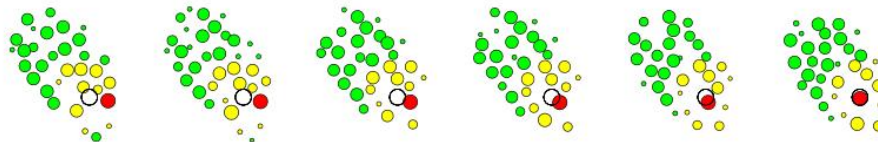


GATCGT  
TCCBIA  
TTACCG  
AGCCTA  
GAACGT  
**Cambridge  
Computational  
Biology  
Institute**

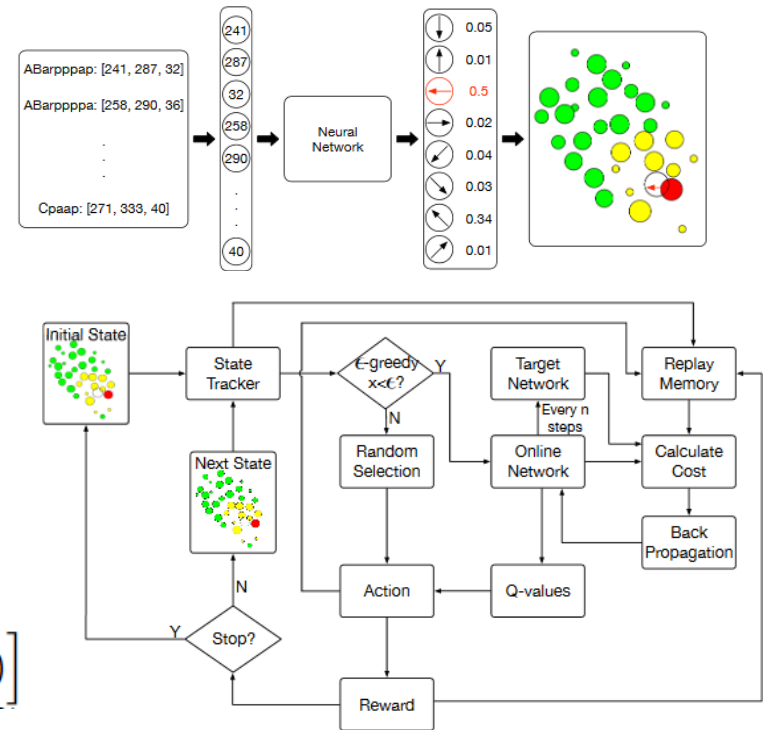
# Deep Q-Network for Cell Movement



(a)



C. Elegans early embryogenesis



$$Q(S_t, A_t | \theta_t) \leftarrow Q(S_t, A_t | \theta_t) + \alpha \left[ R_t + \gamma \max_a Q(S_{t+1}, A_{t+1} | \theta_t^-) - Q(S_t, A_t | \theta_t) \right]$$

successfully developed a cell movement modeling system by integrating deep reinforcement learning with an ABM framework. Our modeling system can learn a cell's optimal path under certain regulatory mechanisms

# Genetic Evolution in Action



## Reprise du learning avec evolution genetique

```
In [28]: 1 deltaNbNeuronList=[40,20,10,10,10,6,4,2,1]
          2
          3 Y_ordrel_option,model_evolution,y_scaler_option,y_scaled_option=CalibrationRestartFromPkgPlus(\
          4     params.LEARNINGBASE_ORIGIN,params.LEARNINGBASE_BUT,deltaNbNeuronList,Ycall_option,y_scaler_option,y_scaled_option,\
          5     "Option",params,generate_errors = False)
          6
```

```
Note = 0.20045304465241123
noteinitiale= 0.20045304465241123
New model synthesized
starting fit
Epoch 1/5000
- 3s - loss: 2.5030e-05 - acc: 2.2222e-06

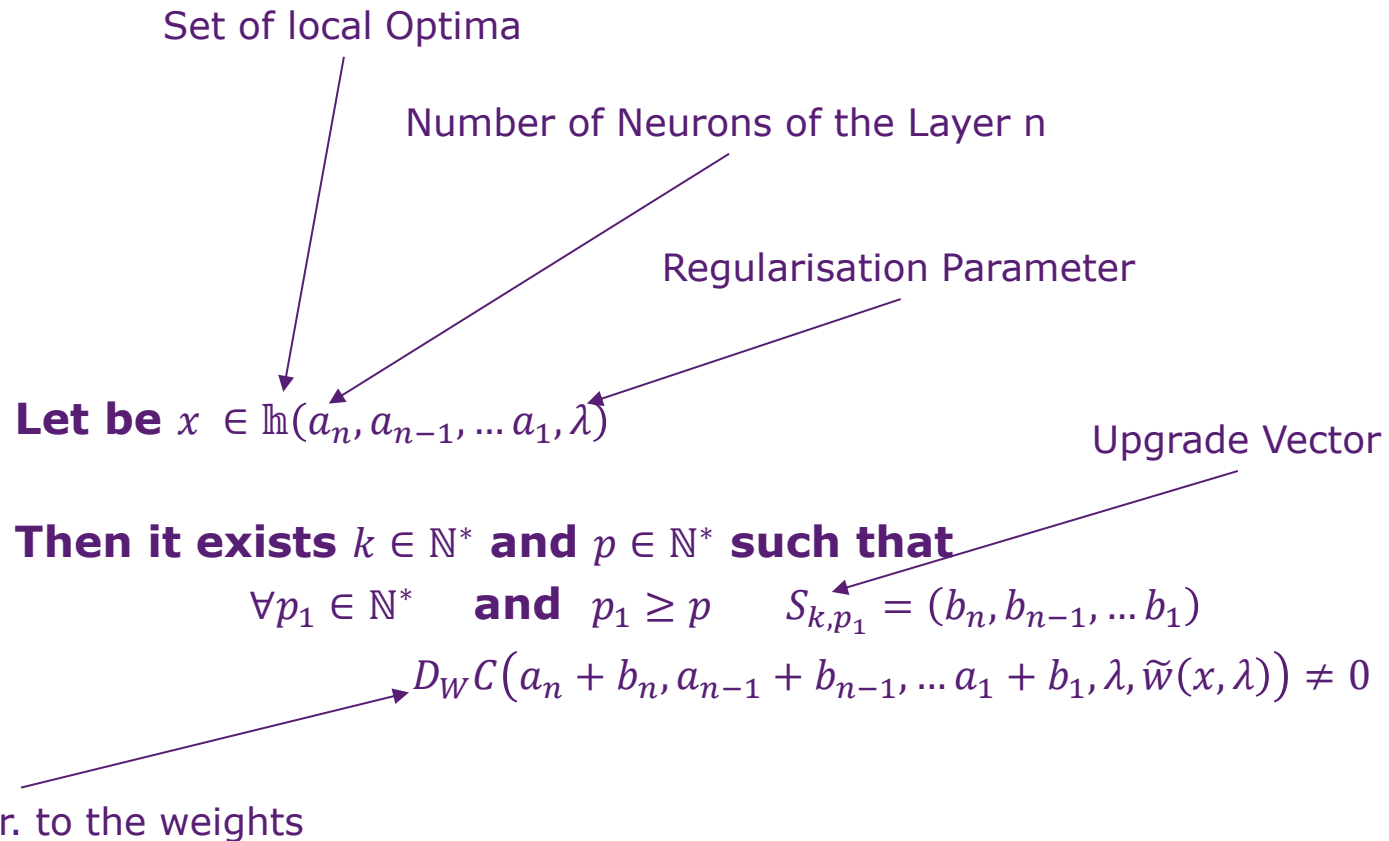
Epoch 00001: loss improved from inf to 0.00003, saving model to BestWeights.hdf5
Epoch 2/5000
- 2s - loss: 1.5327e-06 - acc: 2.2222e-06
```

.....

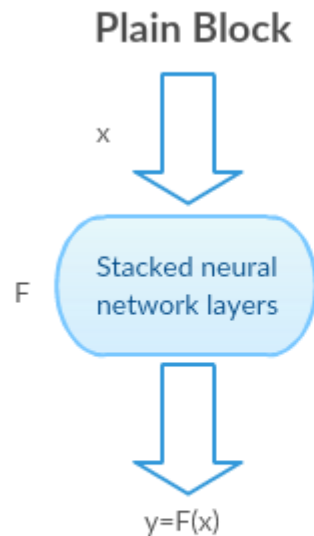
```
Epoch 04999: loss did not improve from 0.00000
Epoch 5000/5000
- 2s - loss: 3.8838e-07 - acc: 2.2222e-06

Epoch 05000: loss did not improve from 0.00000
temps de calibration 11336.246407747269
starting to save
Saved model to disk as Option
number of additional learning : 1
Note = 0.19501618976365487
notefinale= 0.19501618976365487
```

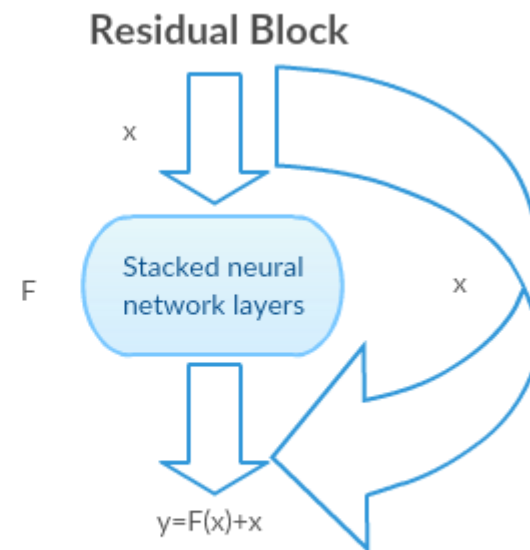
# Upgrade Opportunity and its Monotonicity



# Intuition behind Residual blocks



Hard to get  $F(x)=x$  and make  $y=x$  an identity mapping



Easy to get  $F(x)=0$  and make  $y=x$  an identity mapping

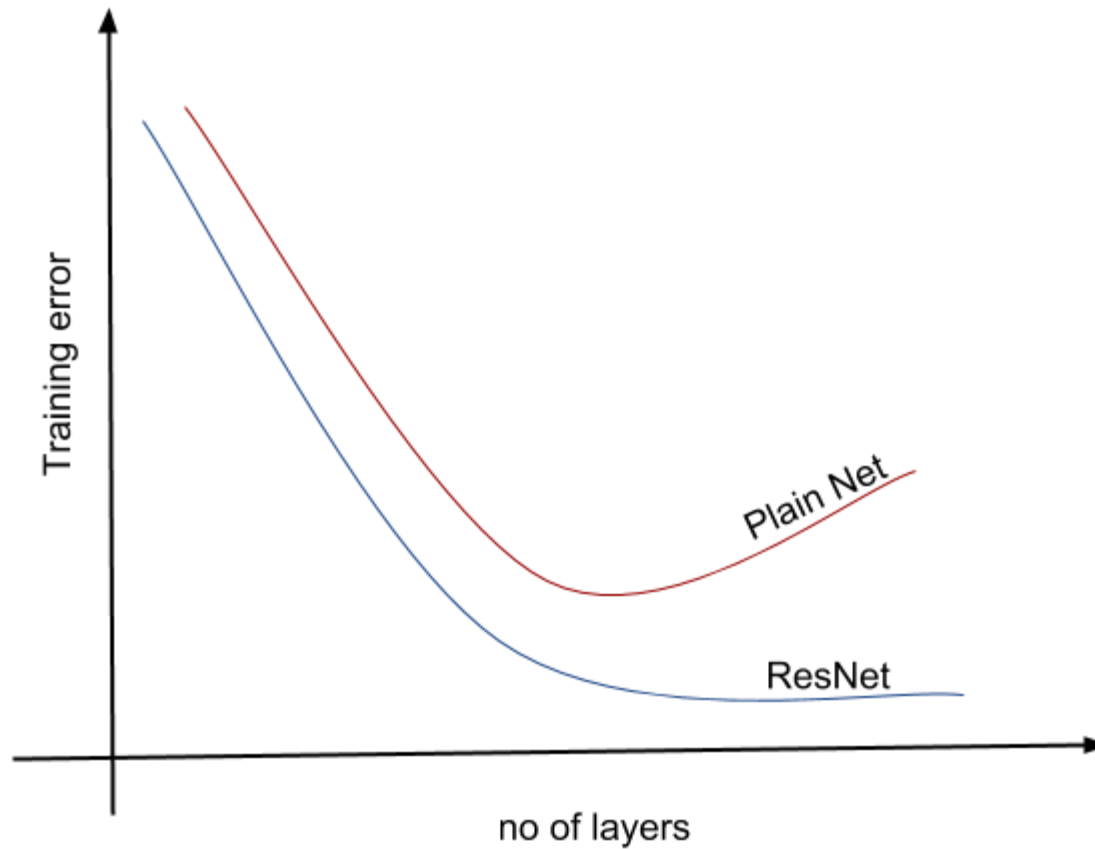


## Was ResNet Successful?

- **Won 1st place in the ILSVRC 2015 classification competition with top-5 error rate of 3.57% (An ensemble model)**
- **Won the 1st place in ILSVRC and COCO 2015 competition in ImageNet Detection, ImageNet localization, Coco detection and Coco segmentation.**
- **Replacing VGG-16 layers in Faster R-CNN with ResNet-101. They observed a relative improvements of 28%**
- **Efficiently trained networks with 100 layers and 1000 layers also.**



# Resnet success



# ILSVRC-2015 results



| method                     | top-1 err.   | top-5 err.        |
|----------------------------|--------------|-------------------|
| VGG [41] (ILSVRC'14)       | -            | 8.43 <sup>†</sup> |
| GoogLeNet [44] (ILSVRC'14) | -            | 7.89              |
| VGG [41] (v5)              | 24.4         | 7.1               |
| PReLU-net [13]             | 21.59        | 5.71              |
| BN-inception [16]          | 21.99        | 5.81              |
| ResNet-34 B                | 21.84        | 5.71              |
| ResNet-34 C                | 21.53        | 5.60              |
| ResNet-50                  | 20.74        | 5.25              |
| ResNet-101                 | 19.87        | 4.60              |
| ResNet-152                 | <b>19.38</b> | <b>4.49</b>       |

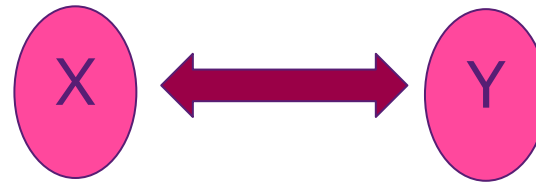
Table 4. Error rates (%) of **single-model** results on the ImageNet validation set (except <sup>†</sup> reported on the test set).

# Take Home Message



- 1) Pricing algorithms have topological characteristics that make them accessible to multidimensional interpolation**
- 2) Handling local optimums during the learning phase can use genetic dynamic learning which involves reinforcement in an essential way. The dynamicity can be made asymptotically efficient .**
- 3) The simplest reinforcement model involves the Q-value associated with each layer. It is a multiarmed bandit type of problem.**
- 4) Exploitation versus exploration analysis shows the  $\epsilon$ -greedy strategy is very good but may develop instabilities that can be fixed with the UCB framework.**

# Interactions Between Layers



Interacting Distrib.

No Interaction Distrib.

Mutual Information

$$I(X; Y) = D_{KL}[p(x, y) || p(x)p(y)]$$

The information improvement compared to an independent case

Distance Between Distributions

# Divergence de Kullback-Leibler



$$D_{KL}[p||q] = \int_{-\infty}^{\infty} p(x) \text{Log} \frac{p(x)}{q(x)} dx$$

$$D_{KL}[p||p] \geq 0 \quad \text{et} \quad D_{KL}(p||p) = 0$$

Mais ce n'est pas une distance car l'inégalité triangulaire n'est pas vérifiée

N'est pas symétrique

Considerant une loi  $q(x)$  définie sur  $E$  et  $F \subset E$

Soit  $q_F(x)$  la loi restreinte à  $F$  (conditionnée par  $F$ )

$$\text{Alors } D_{KL}[q||q_F] = \text{Log} \frac{1}{\text{Prob}[F]}$$

- C'est l'information supplémentaire : conditionnée par  $F$  (en nombre de bits)
- C'est la Rényi divergence pour  $\alpha = 1$ ,  $D_\alpha[p||q] = \frac{1}{1-\alpha} \int_{-\infty}^{\infty} p(x)^\alpha q(x)^{1-\alpha} dx$

Hellinger

La topologie engendrée par  $D_{KL}$  et  $D_\alpha$  ne sont pas les mêmes :  $D_{1/2} \rightarrow \text{Distance}$

$$\lim_{\theta_1 \rightarrow \theta} \frac{1}{(\theta_1 - \theta)^2} D_\alpha[p_{\theta_1}||p_\theta] = \frac{\alpha}{2} J(\theta) \leftarrow \text{Information de Fisher}$$

# KL-Divergence et Entropie Croisée..



Choisir  
Le bon outil



Claude Shannon

$$D_{KL}[p||q] = \int p(x) \log p(x) dx - \int p(x) \log q(x) dx$$

Mesure un écart

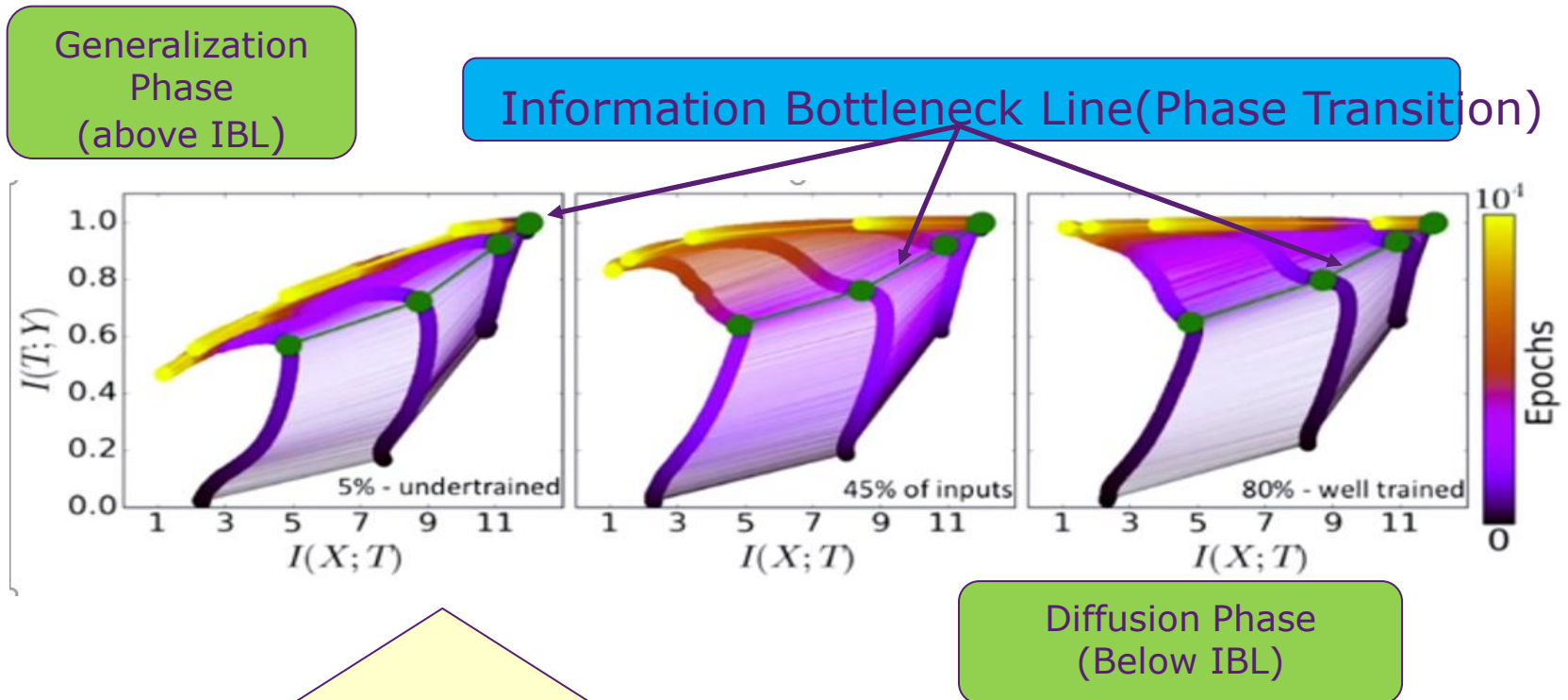
$$D_{KL}[p||q] = H(p) - H(p, q)$$

Entropie

Entropie croisée

- Un critère efficace
- Problèmes de classification
- Gradient lointain encore actif

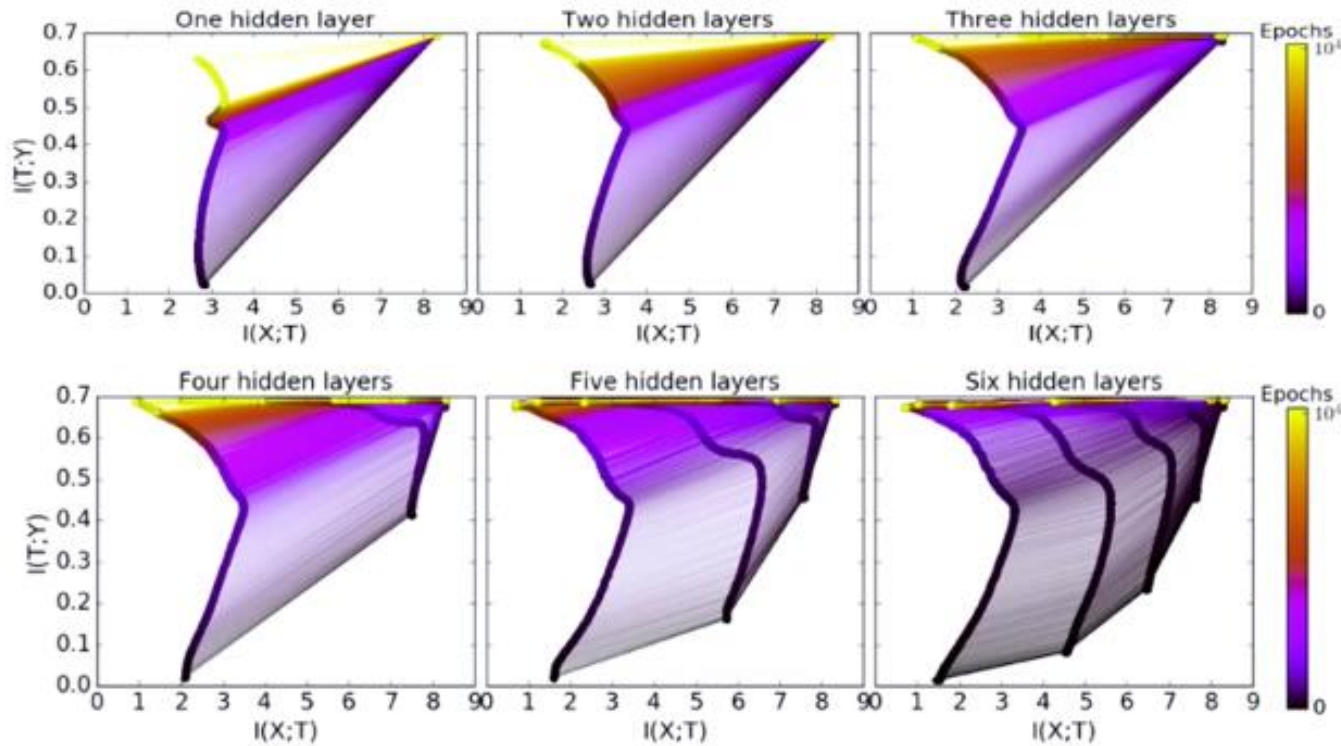
# Information Plane in Action



**Morale:**

You need enough data  
To make Deep Learning Great Again

# Importance of Hidden layers



More layers take much FEWER training epochs for good generalization.

The optimization time depend super-linearly (exponentially?) on the compressed information,  $\Delta I_x$ , for each layer.

**Morale:**  
You need enough Layers  
To make Deep Learning Great Again



# Deep Learning and Scale Analysis



Exemple : Vision

Exemple : Signal Understanding

## HOW A DEEP NEURAL NETWORK SEES

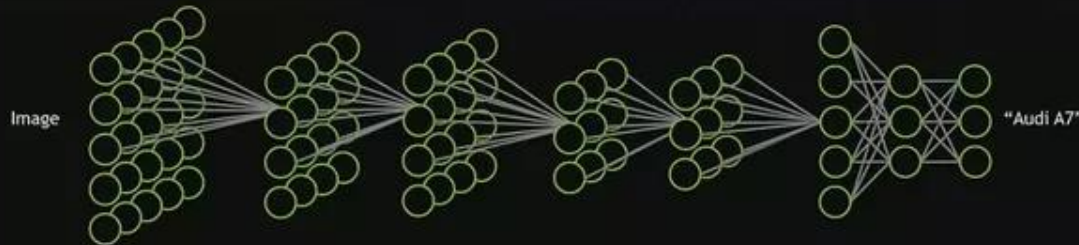
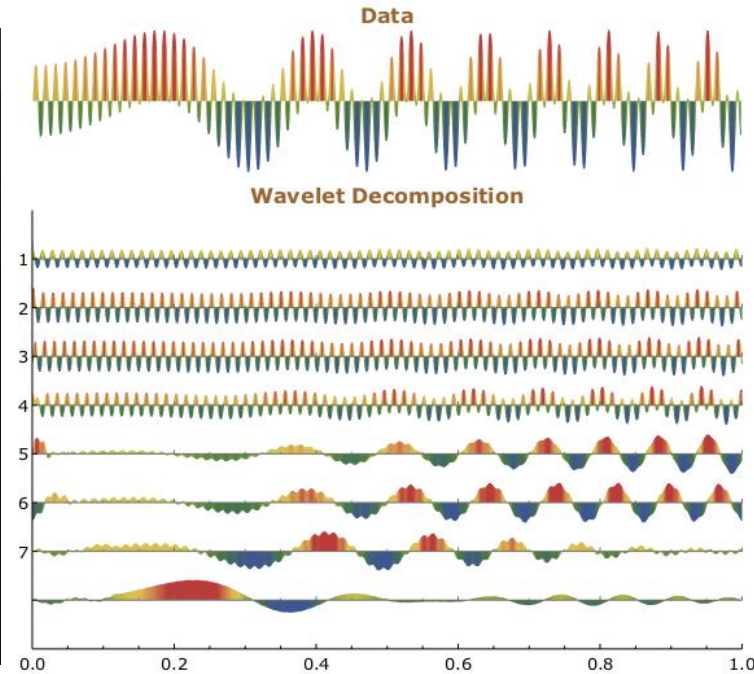


Image source: "Unsupervised Learning of Hierarchical Representations with Convolutional Deep Belief Networks" ICML 2009 & Comm. ACM 2011, Honglak Lee, Roger Grosse, Rajesh Ranganath, and Andrew Ng.

« Scale Dimension »

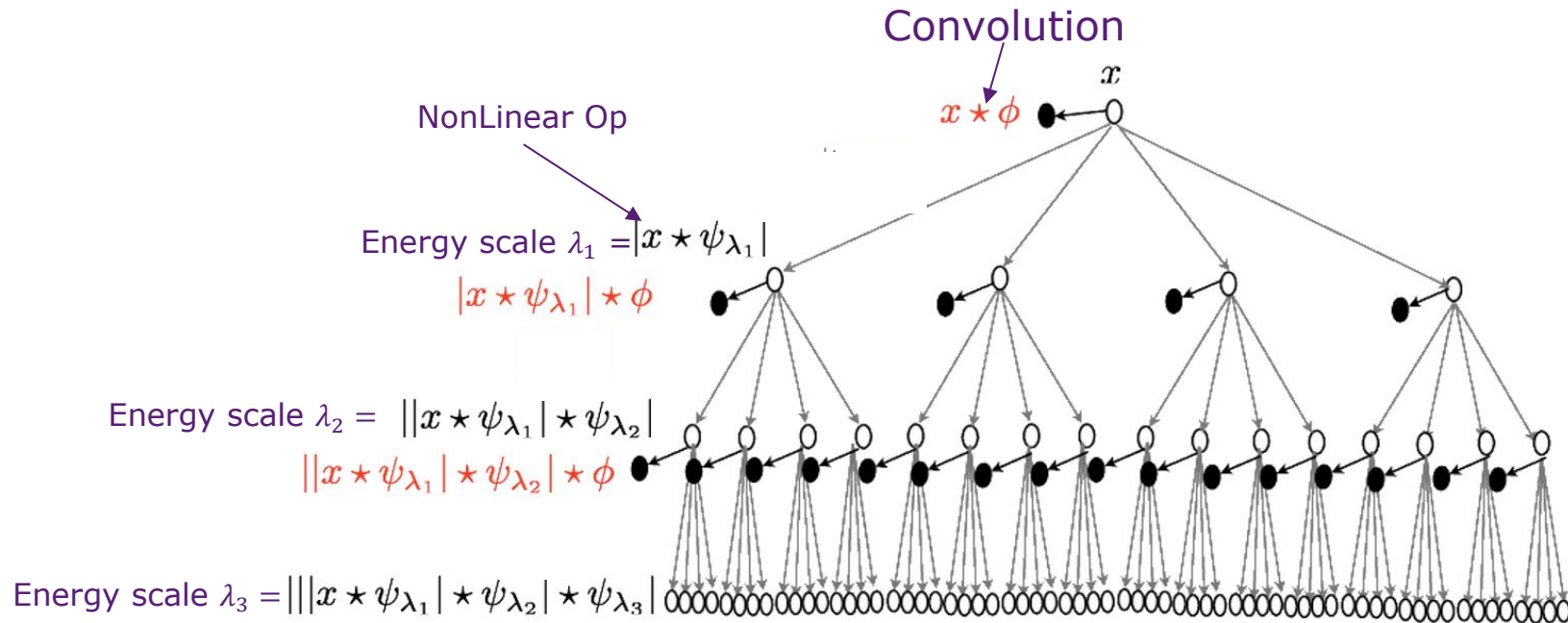


Layer level = scale parameter



order= scale parameter  
Multiscale Wavelet

# At each scale (layer) -> its Energy



Network output:

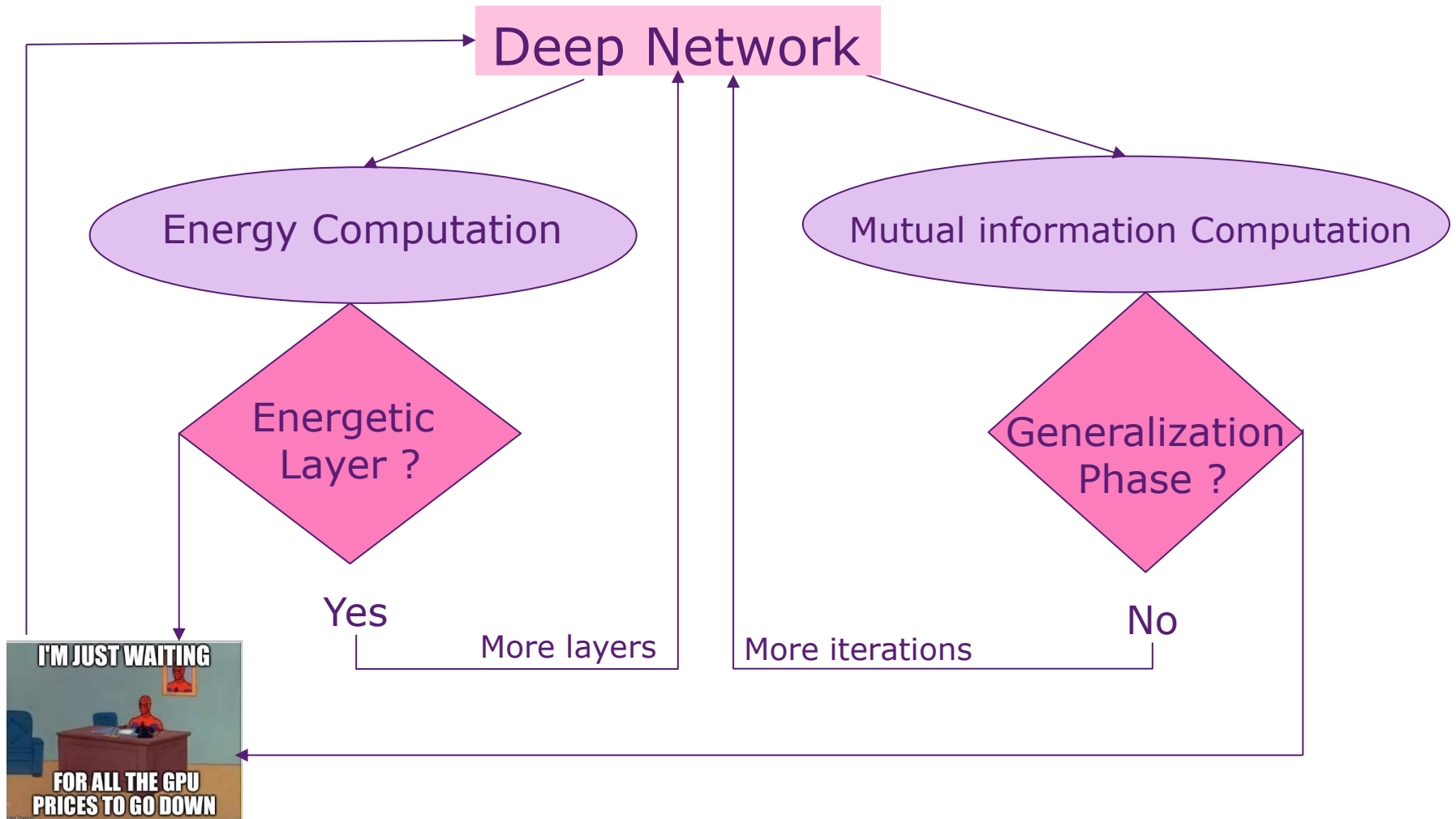
$$Sx = \begin{pmatrix} x \star \phi(u) \\ |x \star \psi_{\lambda_1}| \star \phi(u) \\ ||x \star \psi_{\lambda_1}| \star \psi_{\lambda_2}| \star \phi(u) \\ |||x \star \psi_{\lambda_1}| \star \psi_{\lambda_2}| \star \psi_{\lambda_3}| \star \phi(u) \\ \dots \end{pmatrix}_{u, \lambda_1, \lambda_2, \lambda_3, \dots}$$

## Scattering Transform

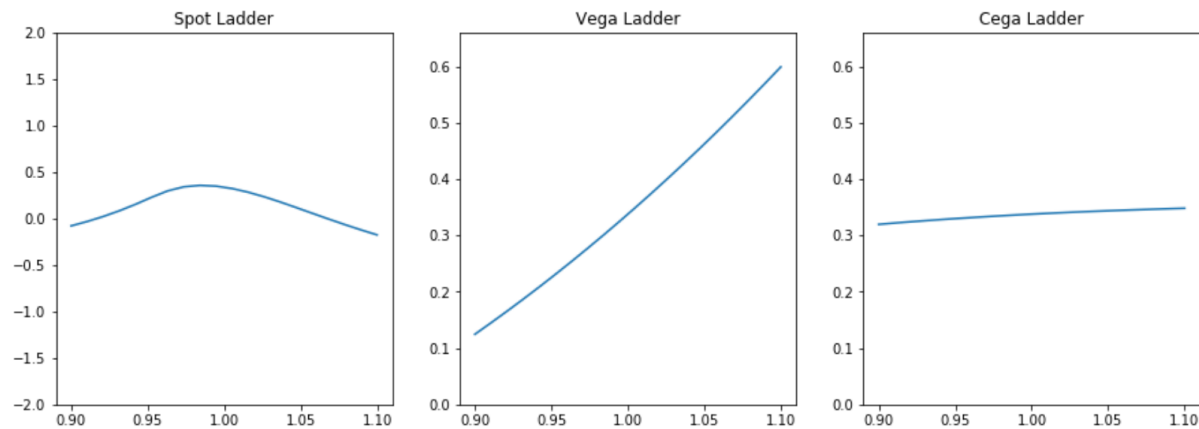
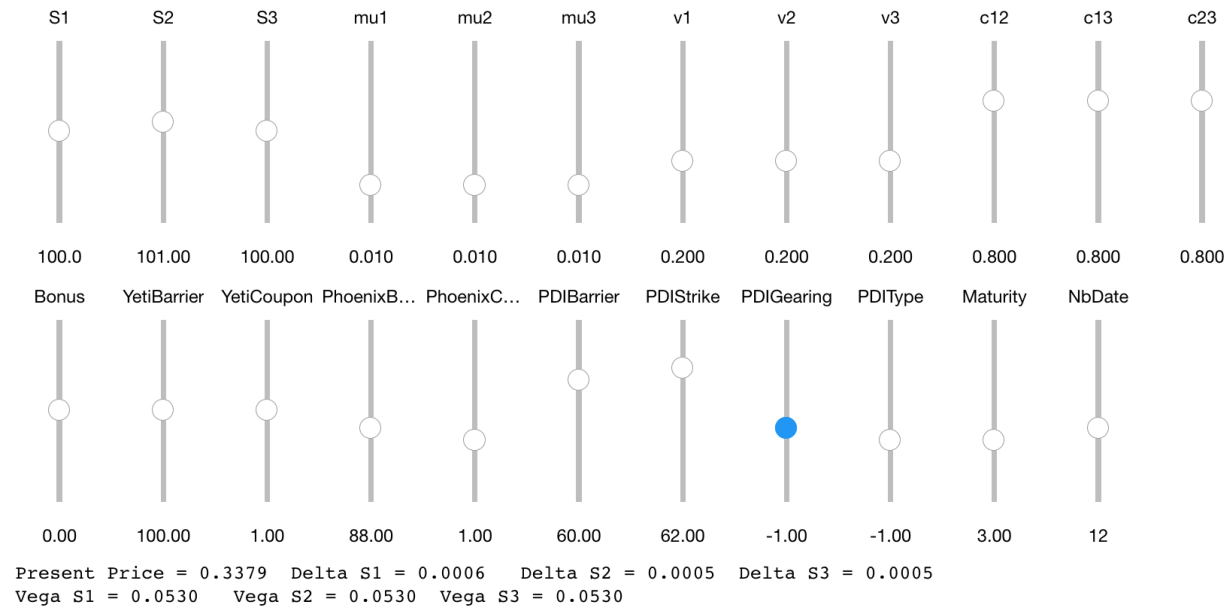
*preserves norms*  $\|Sx\| = \|x\|$

The Wavelet basis can be implicit and optimized

# Use of the Information and Energy



# Smooth Hedge Ratios



# How to get better learning

---



Improving hyper parameters without preserving the learning

Comparing different learning : different lives

Improving hyper parameters while preserving the learning

Evolve a single learning : only one life

# Comparison Multi vs Single



| Multi-Learning                               | Single - Learning                                   |
|----------------------------------------------|-----------------------------------------------------|
| Comparing the performances                   | Monotonicity of the performance improvement         |
| Needs more resources                         | Quicker                                             |
| Better manage the risk of bad starting point | The final results may be handicapped by a bad start |
| Size of the network not necessarily growing  | Size of the network grows with the learning         |
|                                              |                                                     |

# Take Home Message



- 1) Pricing algorithms have topological characteristics that make them accessible to multidimensional interpolation**
- 2) Handling local optimums during the learning phase can use genetic dynamic learning which involves reinforcement in an essential way. The dynamicity can be made asymptotically efficient .**
- 3) Mutual Information brings light to the way each layer behaves during the learning and how the generalization capability develops . Information Plan and Information Bottleneck are key concepts.**
- 4) Energy Analysis brings light to the resolution scale interpretation of layers. It should help deciding the number of needed layers**

# Math 1 Lambda regularized Opt. prob.



## DEFINITION 1

Given a set of linear layers with the RELU activation functions, the layers are described by a set of functions that map the output of the preceding layer to the output of the current layer. Each function is just the activation function, here the RELU function, on the shifted matrix multiplication of the input by the matrix of weights. The cost function is therefore:

$$c(w_{i_N, i_{N-1}}, w_{i_N, i_{N-1}}, \dots, w_{i_1, i_0}) = \sum_{i_N=1}^{I_N} \left( \sigma \left( \sum_{i_{N-1}=1}^{I_{N-1}} w_{i_N, i_{N-1}} \sigma \left( \sum_{i_{N-1}=1}^{I_{N-1}} w_{i_N, i_{N-1}} \dots \sigma \left( \sum_{i_0=1}^{I_0} w_{i_1, i_0} X_{i_0} \right) \right) \right) - Y_{i_N} \right)^2$$

The set of all such neural networks is noted as  $N_{L_n, L_{n-1}, \dots, L_1}$ .

$L_1$  is the set of neurons of layers 1

...

$L_n$  is the set of neurons of layers n

The  $\lambda$ -regularized optimization problem is the problem of minimizing the  $\lambda$ -regularized cost defined by :

$$c_\lambda = c + \lambda \left( \sum_{i_N=0}^{I_N} (w_{i_N, i_{N-1}} - w_{i_N, i_{N-1}}^0)^2 + \dots + \sum_{i_1=0}^{I_1} (w_{i_1, i_0} - w_{i_1, i_0}^0)^2 \right)$$

## Remark 1:

The usual bias is included in the preceding description by having the dimension 0 associated with the bias and having the input of index 0 being always equal to 1.





## THEOREM 1

Given a set of linear layers with the RELU activation functions, it exist a positive number  $\lambda$  such that the  $\lambda$ -regularized optimization problem has a unique global solution. We call it a root lambda. There are a continuous infinity of such lambda. They are all minored by 0, therefore it exists an inferior limit of such a set. We call this limit

## THEOREM 2

The set of root lambda is closed, and then the inferior limit of the set of root lambda is a root lambda.

# Math 3 : Set of Local Optima



## DEFINITION 2

For any  $\lambda \geq 0$  the set of local optima of  $c_\lambda$  is called  $l_\lambda$  . A local optima is given by a couple

$$c_{\lambda, k_\lambda} \equiv \left\{ \{w^{\lambda, k_\lambda}_{i_N, i_{N-1}}, w^{\lambda, k_\lambda}_{i_{N-1}, i_{N-2}}, \dots, w^{\lambda, k_\lambda}_{i_1, i_0}\}, y^{\lambda, k_\lambda} \right\}$$

Where  $k_\lambda$  indexes the set  $l_\lambda : l_\lambda \equiv \{c_{\lambda, k_\lambda}\}_{k_\lambda}$

## THEOREM 3

$U_\lambda = \{y^{\lambda, k_\lambda}\}_{k_\lambda}$  is a finite set.

The set  $l_\lambda$  may not be finite. Therefore  $U_\lambda$  has a maximum element called the  $\lambda$ -global maximum cost . It is associated with a subset of  $l_\lambda$  called the  $\lambda$ -global optima set:  $L_\lambda$

Of course, we have  $L \subset l_\lambda$

# Math 4: Global and Local Optima



## DEFINITION 3

The function  $w : c_{\lambda, k_{\lambda}} \rightarrow w^{\lambda, k_{\lambda}}_{i_N, i_{N-1}}, w^{\lambda, k_{\lambda}}_{i_{N-1}, i_{N-2}}, \dots, w^{\lambda, k_{\lambda}}_{i_1, i_0}$

And  $y : c_{\lambda, k_{\lambda}} \rightarrow y^{\lambda, k_{\lambda}}$

Are called the natural projectors  $w$  and  $y$  of the  $\lambda$ -regularized optimization problem

The sets  $\mathbb{H} = \bigcup_{\lambda \geq 0} L_{\lambda}$  and  $\mathbb{h} = \bigcup_{\lambda \geq 0} l_{\lambda}$  Are dependent on the network

## THEOREM 4

The sets  $\mathbb{H} = \bigcup_{\lambda \geq 0} L_{\lambda}$  and  $\mathbb{h} = \bigcup_{\lambda \geq 0} l_{\lambda}$  are closed under the topology induced by the natural projectors  $w$  and  $y$ .

# Math 5 : Stationarity of Local Optima



We define the functions that associate to a network the set :

$$\mathbb{H} : N_{L_n, L_{n-1}, \dots, L_1}, \lambda \rightarrow \mathbb{H}[L_n, L_{n-1}, \dots, L_1, \lambda]$$

And

$$\mathbb{h} : N_{L_n, L_{n-1}, \dots, L_1}, \lambda \rightarrow \mathbb{h}[L_n, L_{n-1}, \dots, L_1, \lambda]$$

We are interested in increasing the size of the layers when we are stuck in a local minima. The layers being fully connected from one level to the next one are only characterized by their size. So the preceding functions are in fact functions defined on  $\mathbb{N}^n \times \mathbb{R}^+$

$$\mathbb{H} : (a_n, a_{n-1}, \dots, a_1, \lambda) \rightarrow \mathbb{H}(a_n, a_{n-1}, \dots, a_1, \lambda)$$

And

$$\mathbb{h} : (a_n, a_{n-1}, \dots, a_1, \lambda) \rightarrow \mathbb{h}(a_n, a_{n-1}, \dots, a_1, \lambda)$$

For every local optima is associated to a root of a multi-linear equation. The cost function has to be stationary with respect to the variation of the weights.

Let be the cost function associated with the  $\lambda$ -network  $(a_n, a_{n-1}, \dots, a_1, \lambda)$  with the weights  $w$  be

$$C(a_n, a_{n-1}, \dots, a_1, \lambda, w)$$

We assume that  $w(x, \lambda)$  are the weights that cancel the derivative for this point  $x$

The cost being stationary for every local optima we have

So we have  $D_W C(a_n, a_{n-1}, \dots, a_1, \lambda, w(x, \lambda)) = 0 \forall x \in \mathbb{h}(a_n, a_{n-1}, \dots, a_1, \lambda)$

Let be a  $\lambda$ -regularized optimization network  $(a_n, a_{n-1}, \dots, a_1, \lambda, w)$  we can inject this network inside a network with indexes  $\widetilde{a}_k$  and weights  $\widetilde{w}$  such that the cost function at the point  $\widetilde{w}$  is lossless:

$$C(\widetilde{a}_n, \widetilde{a}_{n-1}, \dots, \widetilde{a}_1, \lambda, \widetilde{w}(w)) = C(a_n, a_{n-1}, \dots, a_1, \lambda, w)$$

# Math 6 Existence of global layer upgrade



**For all**  $(\widetilde{a}_n, \widetilde{a}_{n-1}, \dots, \widetilde{a}_1)$  **such**  $\widetilde{a}_k \geq a_k \forall k$

**Let's call this injection the natural**  $(b_n, b_{n-1}, \dots, b_1)$  **–upgrade of the a**  $\lambda$ -**regularized optimization network**  $(a_n, a_{n-1}, \dots, a_1, \lambda, w)$

**Where**  $\forall k \quad b_k = \widetilde{a}_k - a_k \geq 0$

**The purpose of the**  $(b_n, b_{n-1}, \dots, b_1)$  **–upgrade is to get out of the local optima.**

## THEOREM 5

**Let be**  $x \in \mathbb{H}(a_n, a_{n-1}, \dots, a_1, \lambda)$  **and**  $x \notin \mathbb{H}(a_n, a_{n-1}, \dots, a_1, \lambda)$   
**then**

**It exists at least one sequence**  $(b_n, b_{n-1}, \dots, b_1)$  **such that**

$$D_W C(a_n + b_n, a_{n-1} + b_{n-1}, \dots, a_1 + b_1, \lambda, \widetilde{w}(x, \lambda)) \neq 0$$

## Remark 2

**In this theorem and all other following existence theorems, we need to assume that we are not at a global optimum. In practice, we use the cost and there is a value for it, below which we assume a global optima found.**

# Math 7 : Existence of a mono-layer upgrade



## THEOREM 6

**Let be**  $x \in \mathbb{H}(a_n, a_{n-1}, \dots, a_1, \lambda)$  **and**  $x \notin \mathbb{H}(a_n, a_{n-1}, \dots, a_1, \lambda)$   
**then**

**It exists an infinite number of sequence**  $(b_n, b_{n-1}, \dots, b_1)$  **such that**  
 $D_W C(a_n + b_n, a_{n-1} + b_{n-1}, \dots, a_1 + b_1, \lambda, \tilde{w}(x, \lambda)) \neq 0$

## DEFINITION 4

**Let's define the following sequences as “canonical sequences” :**  
 $S_{k,p} = (0, \dots, p, \dots, 0)$  **where p is a the kth position**

## THEOREM 7

**Let be**  $x \in \mathbb{H}(a_n, a_{n-1}, \dots, a_1, \lambda)$  **and**  $x \notin \mathbb{H}(a_n, a_{n-1}, \dots, a_1, \lambda)$   
**then**

**It exists**  $k \in \mathbb{N}^*$  **and**  $p \in \mathbb{N}^*$  **such that**  $S_{k,p} = (b_n, b_{n-1}, \dots, b_1)$  **and**  
 $D_W C(a_n + b_n, a_{n-1} + b_{n-1}, \dots, a_1 + b_1, \lambda, \tilde{w}(x, \lambda)) \neq 0$

# Math 8 Monotonicity of the upgrade opportunity



## THEOREM 8

**Let be**  $x \in \mathbb{h}(a_n, a_{n-1}, \dots, a_1, \lambda)$  **and**  $k \in \mathbb{N}^*$  **and**  $p \in \mathbb{N}^*$

**such that**  $S_{k,p} = (b_n, b_{n-1}, \dots, b_1)$  **and**

$$D_W C(a_n + b_n, a_{n-1} + b_{n-1}, \dots, a_1 + b_1, \lambda, \tilde{w}(x, \lambda)) \neq 0$$

**Then**

$\forall p_1 \in \mathbb{N}^*$  **and**  $p_1 > p$  **such that**  $S_{k,p_1} = (b_n, b_{n-1}, \dots, b_1)$

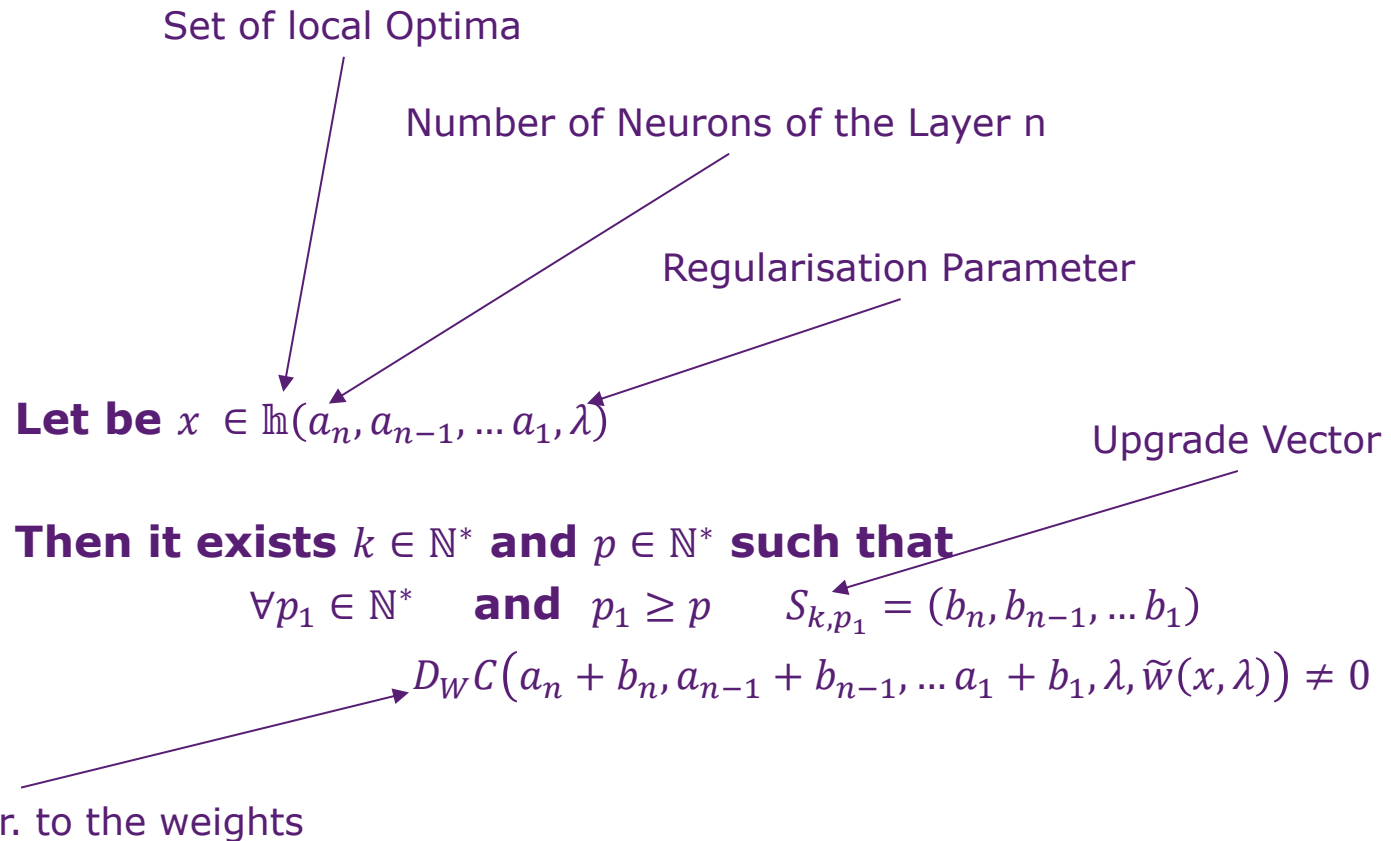
$$D_W C(a_n + b_n, a_{n-1} + b_{n-1}, \dots, a_1 + b_1, \lambda, \tilde{w}(x, \lambda)) \neq 0$$

**Using the preceding theorems, we can describe an algorithm that should allow us to converge toward a global optimum.**

**The global optimum may not be unique.**

**The idea of the algorithm is to explore the different layers with an increasing number of additional neurons. If the optimum is not global, then at a certain level, one of the level should respond that the total differential of the loss should not be 0. We are never sure that the number of additional neuron is enough, but in practice, it works very well.**

# Upgrade Opportunity and its Monotonicity





# Universality of ResNets with Relu



## Identity Matters in Deep Learning

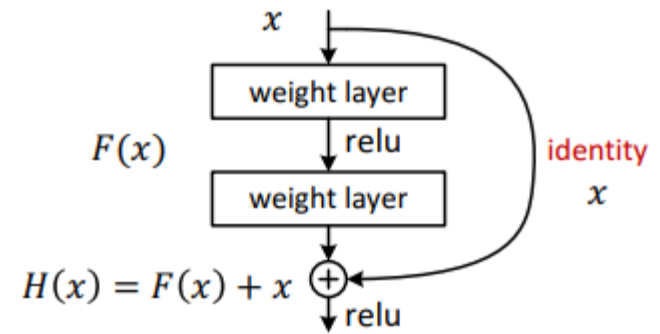
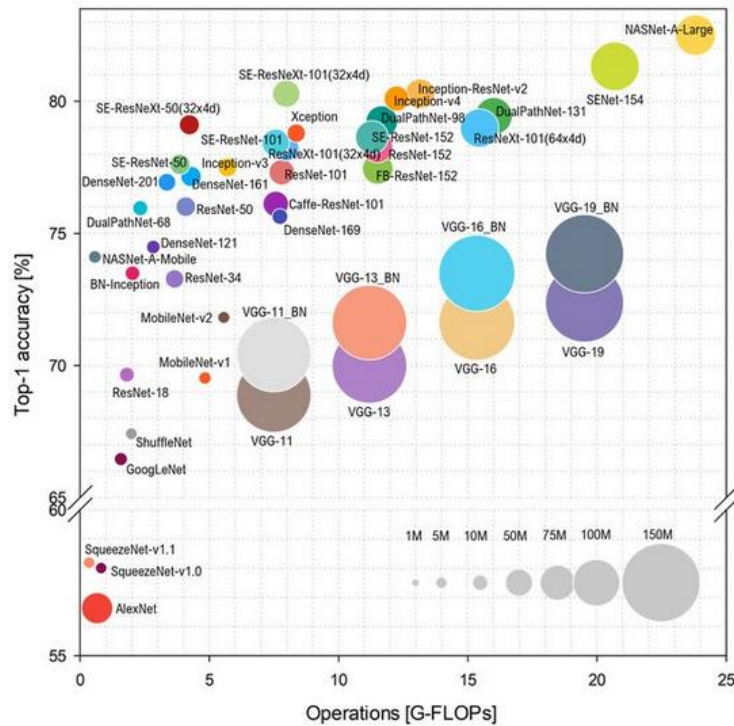
Moritz Hardt\*      Tengyu Ma†

July 23, 2018

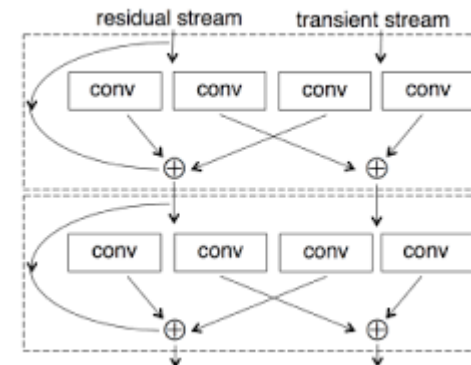
**Assumption 3.1.** *We assume that for every  $1 \leq i < j \leq n$ , we have  $\|x^{(i)} - x^{(j)}\|^2 \geq \rho$  for some absolute constant  $\rho > 0$ .*

**Theorem 3.2.** *Suppose the training examples satisfy Assumption [3.1](#). Then, there exists a residual network  $N$  (specified below) with  $O(n \log n + r^2)$  parameters that perfectly expresses the training data, i.e., for all  $i \in \{1, \dots, n\}$ , the network  $N$  maps  $x^{(i)}$  to  $y^{(i)}$ .*

# ResNets



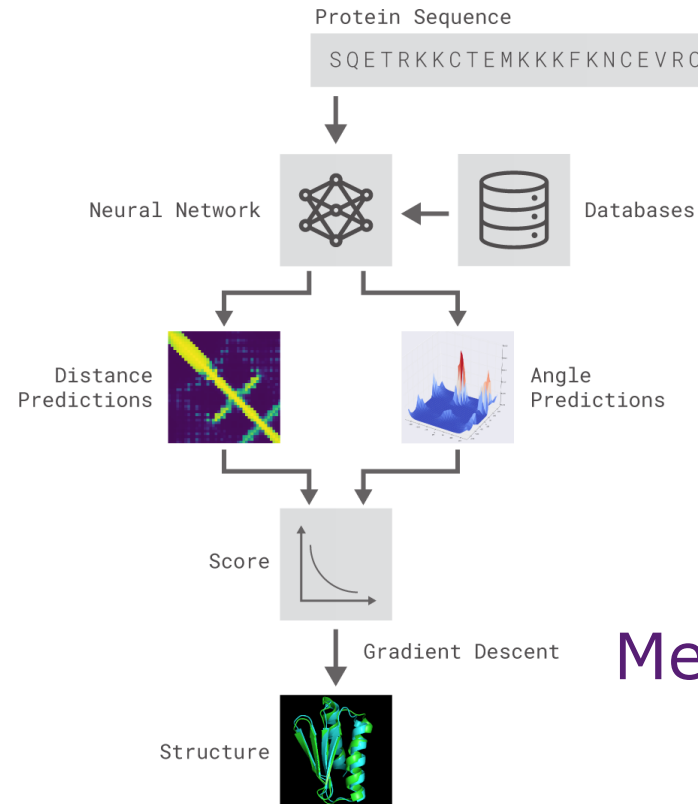
$$H(x) = F(x) + x, \text{ where } F(x) = W2 * \text{relu}(W1 * x + b1) + b2$$



# AlphaFold : Structure



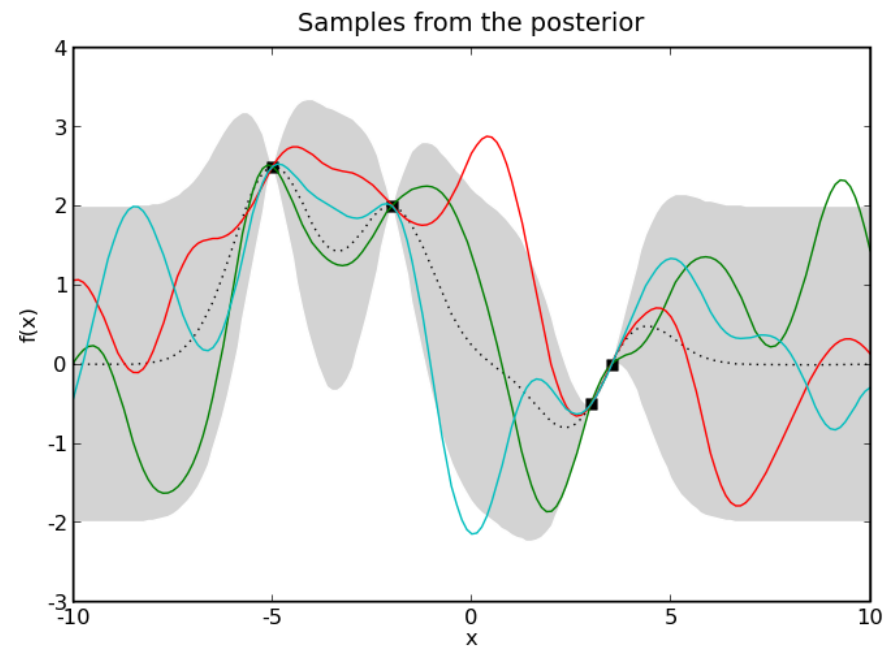
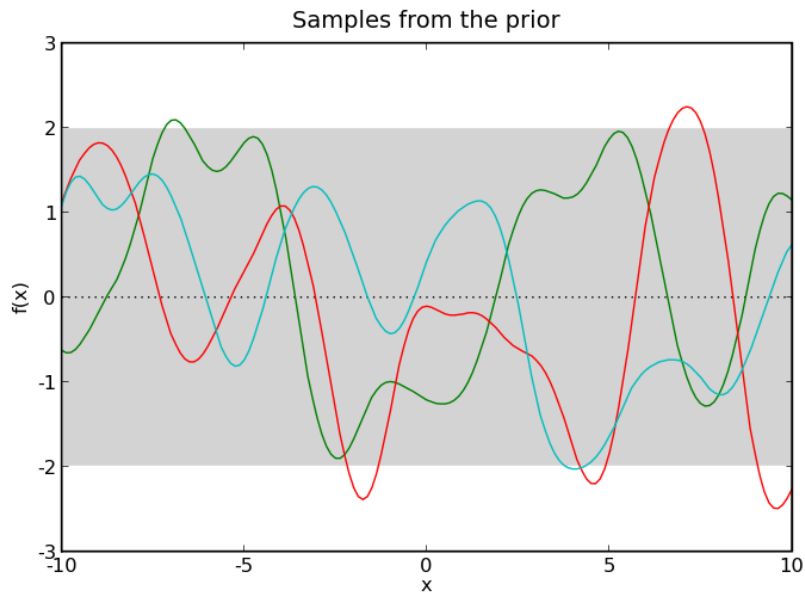
Resnet 1



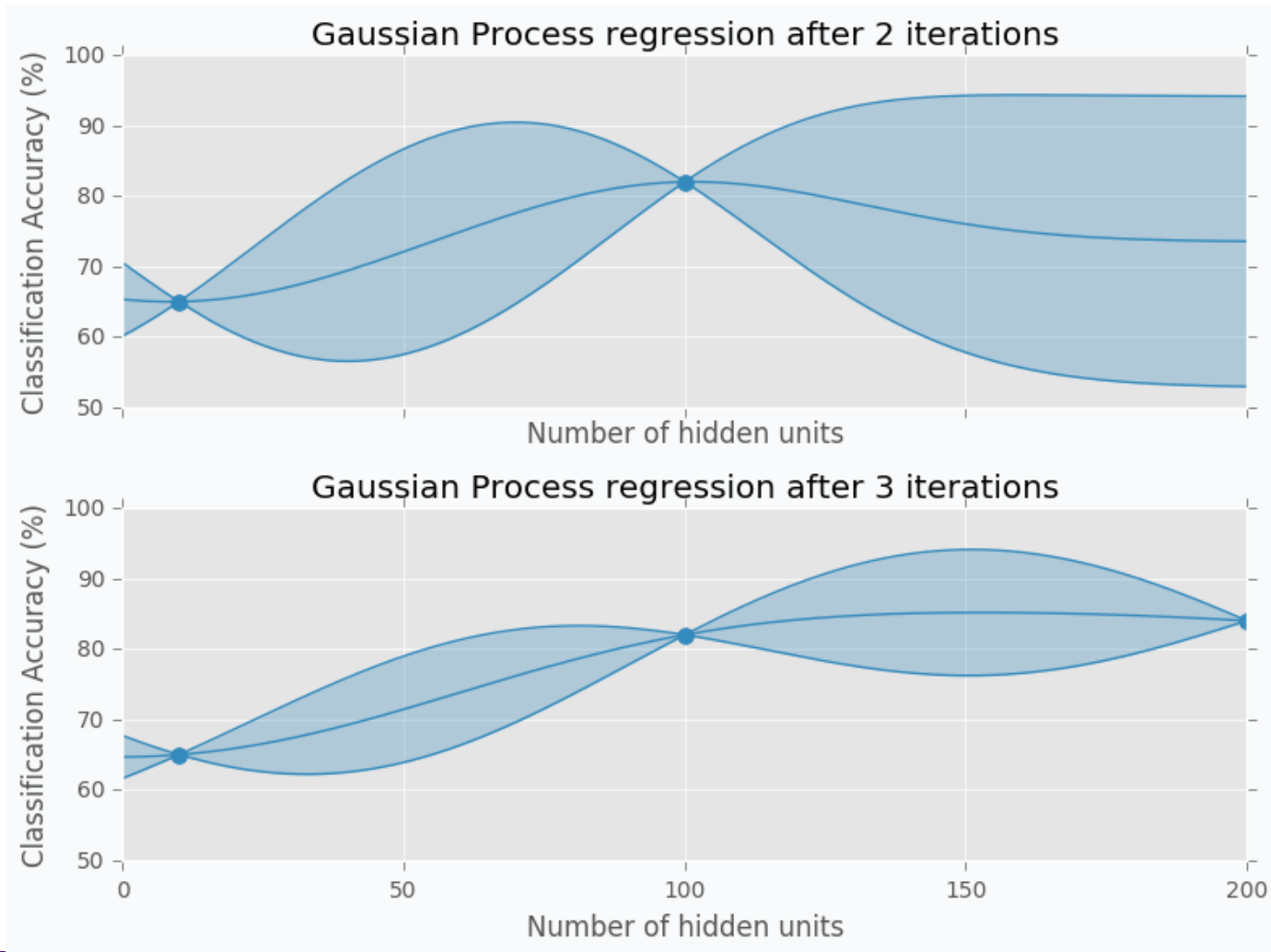
Resnet 2

Meta-Resnet

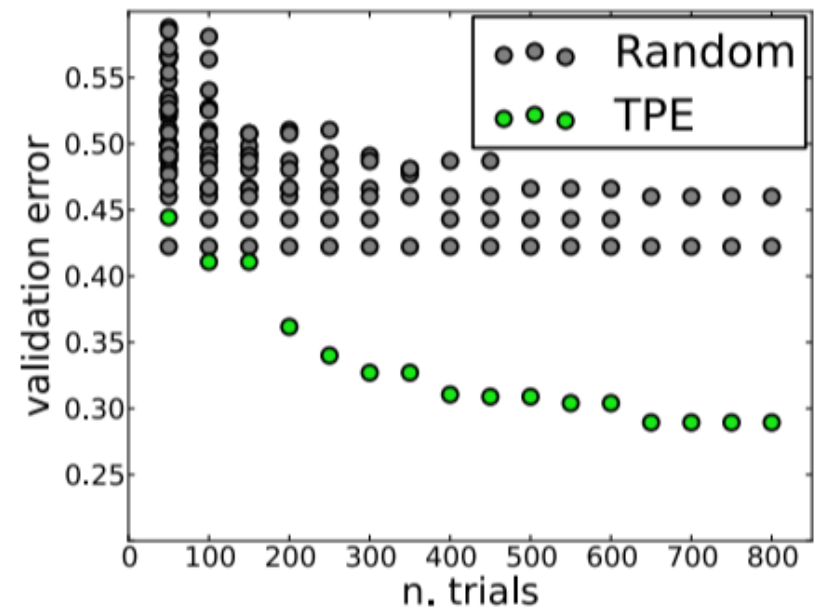
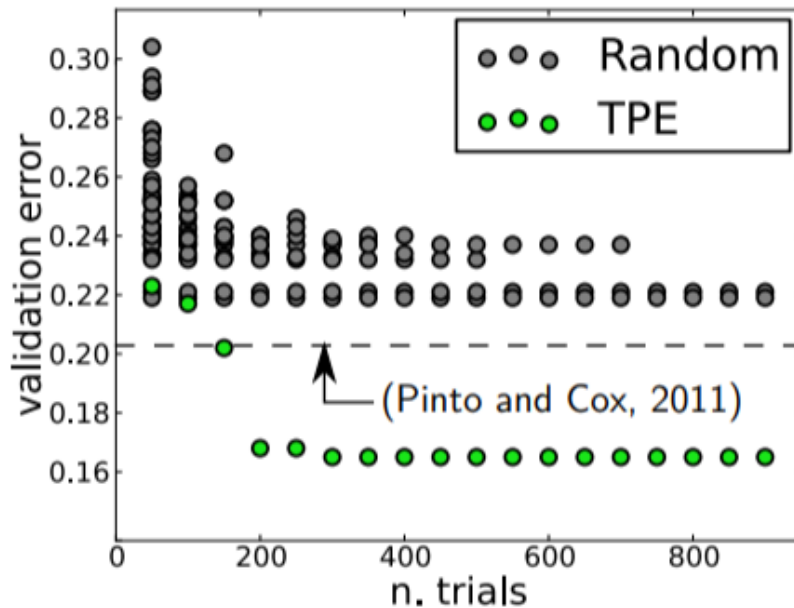
# Gaussian process stability by learning



# Gaussian Process

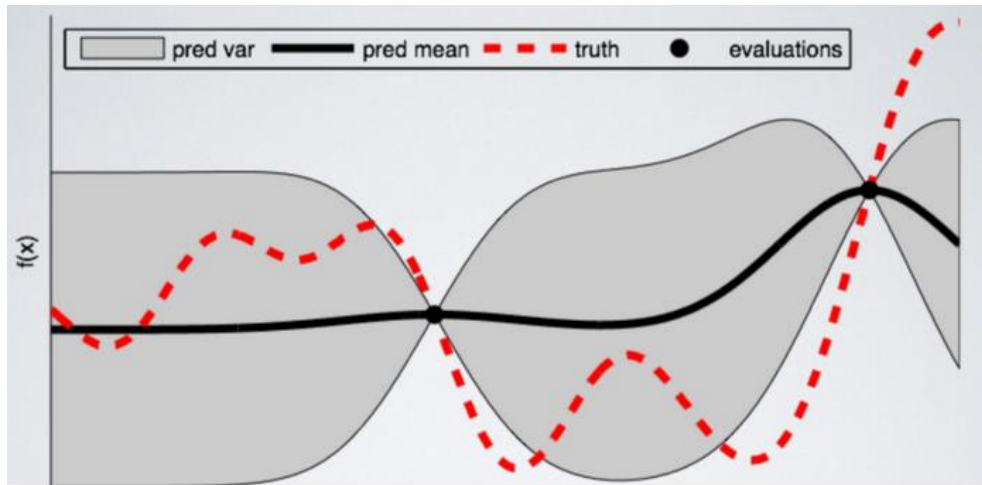


# Comparison Random Search vs TPE



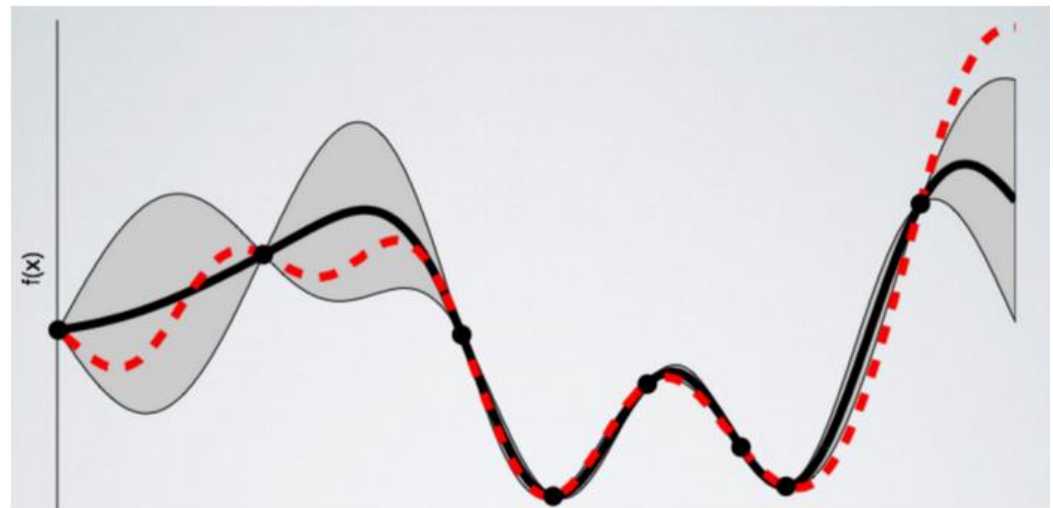
Validation Errors comparing random search and a model based approach on LFW (left) and PubFig83 (right)

# Using a surrogate function



After 2 samples

After 8 samples



# Sequential Model-Based Optimization (SMBO)



- A domain of hyperparameters over which to search
- An objective function which takes in hyperparameters and outputs a score that we want to minimize (or maximize)
- The surrogate model of the objective function
- A criteria, called a selection function, for evaluating which hyperparameters to choose next from the surrogate model
- A history consisting of (score, hyperparameter) pairs used by the algorithm to update the surrogate model

```
SMBO( $f, M_0, T, S$ )  
1    $\mathcal{H} \leftarrow \emptyset,$   
2   For  $t \leftarrow 1$  to  $T$ ,  
3        $x^* \leftarrow \operatorname{argmin}_x S(x, M_{t-1}),$   
4       Evaluate  $f(x^*),$   $\triangleright$  Expensive step  
5        $\mathcal{H} \leftarrow \mathcal{H} \cup (x^*, f(x^*)),$   
6       Fit a new model  $M_t$  to  $\mathcal{H}.$   
7   return  $\mathcal{H}$ 
```

Figure 1: The pseudo-code of generic Sequential Model-Based Optimization.



# selection function of the SMBO



Expected Improvement

$$\text{EI}_{y^*}(x) := \int_{-\infty}^{\infty} \max(y^* - y, 0) p_M(y|x) dy.$$

Here  $y^*$  is a threshold value of the objective function

Maximize the Expected Improvement with respect to  $x$

# Tree-structured Parzen Estimator (TPE)



Instead of directly representing  $p(y|x)$ , it instead uses:

$$p(y|x) = \frac{p(x|y) * p(y)}{p(x)}$$

Bayes Rule in Action!

# Definition of the Expected improvement



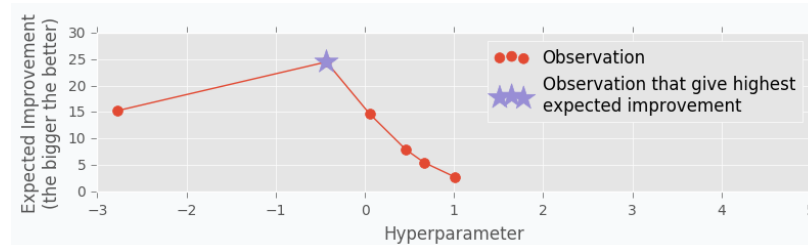
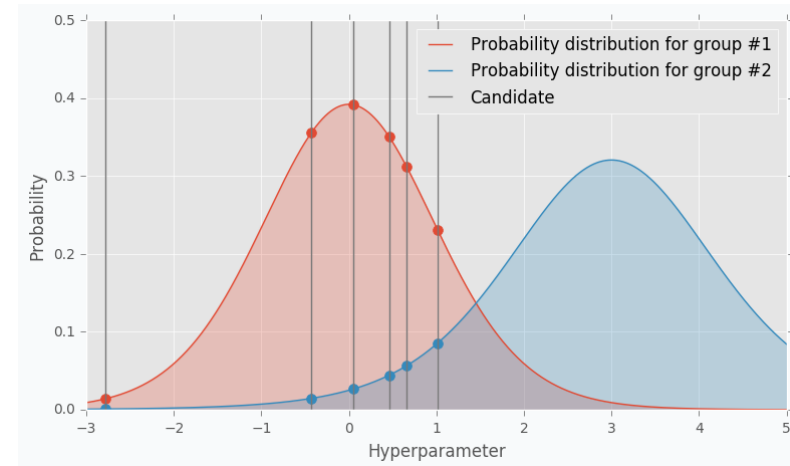
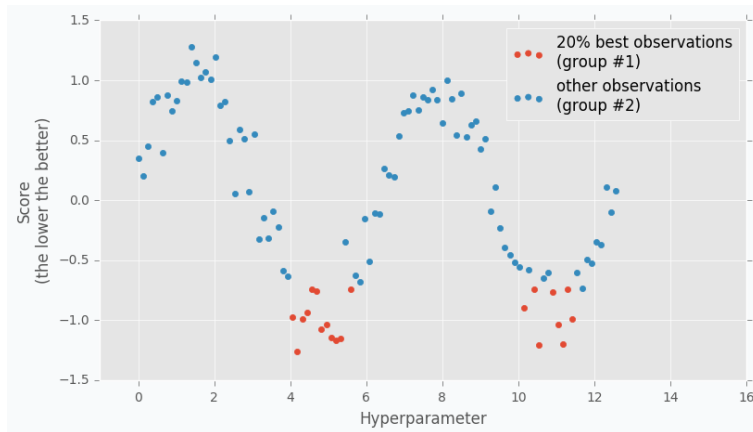
The parametrization of  $p(x, y)$  as  $p(y)p(x|y)$  in the TPE algorithm was chosen to facilitate the optimization of EI

$$EI_{y^*}(x) = \int_{-\infty}^{y^*} (y^* - y)p(y|x)dy = \int_{-\infty}^{y^*} (y^* - y)\frac{p(x|y)p(y)}{p(x)}dy$$

We define by  $y^*$  the quantile  $\gamma$  :  $\gamma = p(y < y^*)$

$$p(x|y) = \begin{cases} \ell(x) & \text{if } y < y^* \\ g(x) & \text{if } y \geq y^* \end{cases}$$

# Choice of f and g functions



# Computation of the Expected improvement



$$p(x) = \int_{\mathbb{R}} p(x|y)p(y)dy = \gamma\ell(x) + (1 - \gamma)g(x).$$

$$\int_{-\infty}^{y^*} (y^* - y)p(x|y)p(y)dy = \ell(x) \int_{-\infty}^{y^*} (y^* - y)p(y)dy = \gamma y^* \ell(x) - \ell(x) \int_{-\infty}^{y^*} p(y)dy,$$

$$EI_{y^*}(x) = \frac{\gamma y^* \ell(x) - \ell(x) \int_{-\infty}^{y^*} p(y)dy}{\gamma \ell(x) + (1 - \gamma)g(x)} \propto \left( \gamma + \frac{g(x)}{\ell(x)}(1 - \gamma) \right)^{-1}$$

# Disadvantages of a TPE



One of the biggest disadvantages of this algorithm is that it selects parameters independently from each other. For instance, there is a clear relation between regularization and number of training epoch parameters. With regularization, we usually can train network for more epochs and with more epochs we can achieve better results. On the other hand without regularization, many epochs can be a bad choice because network starts overfitting and validation error increases. Without taking into account the state of the regularization variable each next choice for the number of epochs can look arbitrary.

# Take Home Message



- 1) Pricing algorithms have topological characteristics that make them accessible to multidimensional interpolation**
- 2) Handling local optimums during the learning phase can use genetic dynamic learning which involves reinforcement in an essential way. The Resnets can help a lot.**
- 3) Mutual Information brings light to the way each layer behaves during the learning and how the generalization capability develops . Information Plan and Information Bottleneck are key concepts.**
- 4) Exploitation versus exploration analysis shows the  $\epsilon$ -greedy strategy is very good but may develops instabilities that can be fixed with the UCB framework.**

*Thanks you for your time*



***We wish you a happy Deep Learning !***

